KU LEUVEN

FACULTY OF ENGINEERING SCIENCE

# Tell Me Why: Eliciting explanations from AI models

Ștefan-Cătălin Cucu

Thesis submitted for the degree of Master of Science in Engineering: Computer Science, option Artificial Intelligence

*Supervisor*
Prof. dr. ir. Wouter Joosen
Prof. dr. ir. Lieven Desmet

*Assessor*
Vera Rimmer
Giuseppe Garofalo

*Assistant-supervisor*
Ilias Tsingenopoulos
Vera Rimmer

Academic year 2025 – 2026

© 2026 KU Leuven – Faculty of Engineering Science
Published by Ștefan-Cătălin Cucu,
Department of Computer Science, Celestijnenlaan 200A bus 2402, B-3001 Leuven

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher. This publication contains the study work of a student in the context of the academic training and assessment. After this assessment no correction of the study work took place.

# Contents

**Abstract** iii

**List of Figures and Tables** iv

**List of abbreviations and symbols** vi

**1 Introduction** 1
- 1.1 Motivation . . . 1
- 1.2 Research Goal . . . 2
- 1.3 Thesis Structure . . . 2

**2 Background and Related Work** 5
- 2.1 Dynamic Malware Analysis . . . 5
- 2.2 Explainable Artificial Intelligence . . . 5
- 2.3 Adversarial Robustness . . . 7
- 2.4 Related Work . . . 8

**3 Dataset and Baseline classifiers** 11
- 3.1 Dataset . . . 11
- 3.2 Baseline Classifiers . . . 15

**4 Robustness and Explainability** 19
- 4.1 Explainability Analysis . . . 19
- 4.2 Robustness as a Working Hypothesis . . . 26

**5 LLM-Based Adversarial Evaluation** 27
- 5.1 Introduction . . . 27
- 5.2 Task and Threat Model . . . 27
- 5.3 Initial Adversarial Data . . . 29
- 5.4 Supervised Fine-Tuning . . . 30
- 5.5 GRPO . . . 30
- 5.6 Evaluation & results . . . 32
- 5.7 Generated Changes . . . 33
- 5.8 Limitations . . . 35

**6 Scaling the Adversarial Evaluator** 37
- 6.1 Introduction . . . 37
- 6.2 Multi-Turn Feedback Loop . . . 37
- 6.3 Training . . . 38
- 6.4 Evaluation . . . 38

6.5 Robustified Classifier . . . . . 40

**7 Generating Explanations** **43**
7.1 Introduction . . . . . 43
7.2 Teacher Explanation Data . . . . . 43
7.3 Initial Supervised Fine-Tuning . . . . . 44
7.4 Fusion Architecture . . . . . 45
7.5 Training . . . . . 48

**8 Evaluation and Discussion** **51**
8.1 Quantitative Evaluation . . . . . 51
8.2 Qualitative Evaluation . . . . . 55
8.3 Discussion . . . . . 58

**9 Conclusions** **59**

**A Experimental Material** **63**
A.1 Prompts . . . . . 63

**Bibliography** **67**

**Code of Conduct** **72**

# Abstract

Machine-learning models can classify malware with high accuracy while providing little information about why a particular decision was made. For a security analyst, a confidence score alone is difficult to inspect or trust without knowing which observed behaviors support it. This thesis investigates how malware classification from dynamic sandbox reports can be made more explainable and robust, with the ultimate goal of producing useful natural-language explanations of classifier decisions.

A neural classifier is first analyzed using feature-attribution and perturbation methods, revealing both meaningful behavioral evidence and reliance on dataset-specific artifacts. A language-model-based adversarial evaluator is then developed to generate behavior-preserving modifications that expose weaknesses in the classifier and provide additional examples for adversarial hardening. Finally, the classifier is combined with a language model through a learned fusion architecture. The classifier remains responsible for the prediction, while its decision and internal representations condition the generation of an analyst-readable explanation.

The experiments show that this separation is effective: the dedicated classifier retains strong predictive performance, while classifier-conditioned explanations are more grounded, useful, and faithful than explanations produced by a language model alone. The results demonstrate that natural-language explanation can provide a practical interface for inspecting malware-classification decisions rather than treating them as opaque confidence scores.

# List of Figures and Tables

## List of Figures


3.1 Number of reports in the local AutoRobust dataset, counted during the audit in this thesis. . . . 12
3.2 Presence of each behavioral category by class and malware family. . . . 13
3.3 Token length distribution of the reports. The vertical marker shows the 32,768 token context limit. . . . 14
3.4 Training history of the selected MLP. . . . 16
3.5 Test macro-F1 with bootstrap intervals (left) and the MLP confusion matrix (right). . . . 17
3.6 Test accuracy for goodware and each malware family. . . . 18

4.1 Category-ablation effects. . . . 21
4.2 Local SHAP explanation. . . . 23
4.3 Entry-level Integrated Gradients. . . . 24

5.1 GRPO reward and KL divergence. . . . 32
5.2 Evasion rates. . . . 33
5.3 Best goodware-score distributions. . . . 34
5.4 Effective additions by category. . . . 34

6.1 Multi-turn feedback loop. . . . 38
6.2 Evasions by turn. . . . 39
6.3 Classifier-score trajectories. . . . 40
6.4 Perturbation-budget distributions. . . . 41
6.5 Clean classification performance before and after adversarial hardening. 42
6.6 Classification accuracy by class and malware family before and after adversarial hardening. . . . 42

7.1 Classifier–LLM fusion architecture. . . . 46
7.2 Fusion-training diagnostics. . . . 49

8.1 Classification scores. . . . 52
8.2 Prediction counts. . . . 53

# List of Tables

3.1 Composition of the local AutoRobust dataset used in this thesis. . . . . 12
3.2 Composition of the duplicate-safe experimental splits. . . . . 15
3.3 Hyperparameters selected on the validation set. . . . . 16
3.4 Performance on the duplicate-safe test set. . . . . 17
3.5 Accuracy by class and malware family. . . . . 17

4.1 Explainability settings. . . . . 20
4.2 Category-ablation statistics. . . . . 21
4.3 Largest local SHAP contributions. . . . . 22
4.4 Prevalence of SHAP-selected entries. . . . . 23

5.1 Permitted AutoRobust transformations [37]. . . . . 28
5.2 SFT configuration. . . . . 31
5.3 GRPO configuration. . . . . 32
5.4 Mean additions by category. . . . . 34

6.1 Multi-turn evasion results. . . . . 39
6.2 Final perturbation budgets. . . . . 40
6.3 Clean classification before and after adversarial hardening. . . . . 41

8.1 Judge scores. . . . . 53
8.2 Explanation faithfulness under deletion-based perturbation. . . . . 54

# List of abbreviations and symbols

## Abbreviations

| | |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| BF16 | Brain Floating Point 16-bit format |
| CoT | Chain of Thought |
| DLL | Dynamic-Link Library |
| F1 | Harmonic mean of precision and recall |
| GNN | Graph Neural Network |
| GPU | Graphics Processing Unit |
| GRPO | Group Relative Policy Optimization |
| IG | Integrated Gradients |
| JSON | JavaScript Object Notation |
| KL | Kullback–Leibler |
| LIME | Local Interpretable Model-agnostic Explanations |
| LLM | Large Language Model |
| LoRA | Low-Rank Adaptation |
| ML | Machine Learning |
| MLP | Multilayer Perceptron |
| RAM | Random-Access Memory |
| RL | Reinforcement Learning |
| ROC-AUC | Area Under the Receiver Operating Characteristic Curve |
| SFT | Supervised Fine-Tuning |
| SHAP | SHapley Additive exPlanations |
| VRAM | Video Random-Access Memory |
| XAI | Explainable Artificial Intelligence |

# Chapter 1

# Introduction

## 1.1 Motivation

With the growing use of machine learning and, more recently, large language models, AI decision-making has become increasingly common across many domains, including cybersecurity and malware detection. Before, malware detection relied on signatures, comparing unknown programs against patterns extracted from identified malware. However, these approaches are less effective against prviously unseen or obfuscated variants [17, 9]. As such, modern malware detection relies more often on machine-learning models based on behavioral analysis. While these learned models can achieve strong predictive performance, the reasoning for their decision is considerably less transparent than explicit rules or signatures - a classifier may produce a label without providing an immediately interpretable account of which evidence drove that decision [8, 6].

A malware classifier can therefore produce a very confident prediction while still leaving an important question unanswered: *why?* For an analyst, a score such as "99% malware" says that the model is confident, but not which observed behaviors caused that confidence or whether those behaviors provide meaningful evidence of malicious activity.

A high accuracy on a test dataset does not guarantee that a classifier has learned the real distinction between what is malicious and not. Machine-learning models can exploit *shortcuts*: features that correlate strongly with the target label in the training data but do not represent the underlying concept that should determine the prediction [12].

In malware analysis, this can mean assigning importance to a particular filename, registry value or other artifact because it happens to occur frequently in one part of the dataset. Previous work has shown that malware classifiers can be vulnerable to feasible modifications of behavioral reports and that these vulnerabilities can be connected to spurious correlations learned from the data [37].

While such a model can output impressive performance metrics, if changing an incidental artifact can flip the classifier prediction without changing the malicious behavior, the prediction is both difficult to trust and potentially easy to evade.

As such, a malware classifier should be both explainable and robust. Explainability shows which parts of a report influence the prediction, while adversarial attacks can test how stable that decision is when those parts are changed.

Current explainability methods, however, do not necessarily produce explanations that are useful to a human analyst. Methods such as SHAP or Integrated Gradients can identify influential features, but their output is typically a collection of attribution scores rather than an explanation of what the observed behavior means or why it supports a malware decision. Cybersecurity analysts often need high level explanations that connect the relevant evidence that justifies the result [33, 23]. This motivates the final part of this thesis: using a language model to turn the classifier's decision and internal representation into a natural-language explanation of the observed behavior.

The overall goal is therefore not only to build a classifier that separates malware from goodware, but to investigate what it has learned, how easily its decision can be manipulated, and whether that decision can ultimately be presented in a form that a human can understand.

## 1.2 Research Goal

The goal of this thesis is to investigate how decisions made by a malware classifier over dynamic-analysis reports can be made more inspectable to a human analyst. This includes studying what information the classifier relies on, whether its decisions are robust to behavior-preserving transformations, and whether a language model can translate the classifier decision and representation into a useful natural-language explanation.

More specifically, I address the following research questions:

1. What can current explainability methods reveal about the behavior and shortcuts used by a malware classifier?
2. Can a language-model adversarial attacker expose weaknesses in the classifier using transformations of behavioral reports?
3. Can a language model be combined with a dedicated classifier to generate useful and faithful natural-language explanations?

## 1.3 Thesis Structure

Chapter 2 introduces the necessary background and related work on dynamic malware analysis, explainability, adversarial robustness, and language-model training. Chapter 3 describes the dataset and establishes the baseline malware classifiers. Chapter 4 analyzes the selected classifier using post-hoc explainability methods. Chapters 5 and 6 develop the LLM-based adversarial evaluator, progressing from single-turn attacks to iterative feedback. Chapter 7 introduces the classifier–LLM fusion architecture. Chapter 8 evaluates the complete system quantitatively and

qualitatively and discusses the implications of the results. Finally, Chapter 9 summarizes the conclusions and directions for future work.

# Chapter 2

# Background and Related Work

## 2.1 Dynamic Malware Analysis

Malware can be analyzed either statically, by inspecting a program without executing it, or dynamically, by observing its behavior while it runs. Dynamic analysis exposes the runtime behavior of the program and can provide evidence that is less dependent on its exact binary representation [37].

Dynamic analysis is performed inside a sandbox: an isolated environment in which a program is executed while its interactions with the operating system are recorded. The resulting behavior report can contain ample observations, ranging from files accessed and commands executed, to resolved APIs and accessed registry values. These reports can be large and cumbersome to inspect manually, which provides a strong motivation for automating their analysis with machine-learning models [36, 37].

Machine-learning approaches have therefore been used to learn directly from observed program behavior, allowing a classifier to combine information from many different events rather than relying on individual predefined indicators [28, 36]. However, behavioral classifiers can still exploit spurious correlations in the collected reports. In the AutoRobust setting, Tsingenopoulos et al. showed that such correlations can make otherwise accurate malware classifiers vulnerable to feasible modifications of their behavioral input [37]. This connection between learned behavior and classifier robustness is particularly relevant to the adversarial evaluation developed later in this thesis.

## 2.2 Explainable Artificial Intelligence

Explainable Artificial Intelligence studies methods for making the predictions of machine-learning models more understandable to humans. An explanation can be global, describing the general behavior of a model, or local, describing why the model produced a particular prediction. We will primarily deal with local explanations, since the goal is to identify which parts of an individual report influence its malware classification.

This work uses XAI methods based on feature attribution. Given a model $f$ and an input $x$, an attribution method assigns a score to each input feature indicating its contribution to the prediction. For us, features consist of behavioral report entries.

### 2.2.1 SHAP

SHAP (SHapley Additive exPlanations) applies Shapley values from cooperative game theory to feature attribution [20]. For a set of features $F$, the Shapley value of feature $i$ is

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[ f(S \cup \{i\}) - f(S) \right], \tag{2.1}$$

where $S$ is a subset of features not containing $i$. The difference $f(S \cup \{i\}) - f(S)$ measures the change in prediction obtained by adding feature $i$ to that subset. The Shapley value averages this marginal contribution over all possible subsets, weighted so that different coalition sizes are represented appropriately.

Computing all possible coalitions becomes infeasible as the number of features grows. Kernel SHAP therefore approximates the Shapley values by evaluating a sample of feature coalitions and fitting a weighted linear surrogate model [20].

### 2.2.2 Integrated Gradients

For differentiable models, feature importance can also be derived from gradients, as a gradient at the input describes the local sensitivity of the prediction. Integrated Gradients implements this by accumulating gradients along a path between a reference input $x'$ and the input $x$ [35].

For input dimension $i$, the attribution is defined as

$$\mathrm{IG}_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial f\left(x' + \alpha(x - x')\right)}{\partial x_i} \, d\alpha, \tag{2.2}$$

where $\alpha$ moves from the reference $x'$ to the actual input $x$. In practice, the integral is approximated using a finite number of interpolation steps [35].

Integrated Gradients satisfies a completeness property: under the assumptions of the method, the sum of the individual attributions corresponds to the difference between the model output for the input and for the reference,

$$\sum_i \mathrm{IG}_i(x) \approx f(x) - f(x'). \tag{2.3}$$

The reference input is therefore an important part of the explanation. Attributions describe how the prediction changes relative to this reference, rather than providing an absolute measure of feature importance.

### 2.2.3 Perturbation and ablation

A more direct way of measuring importance is to modify the input and observe how the prediction changes. For an input $x$ and a feature or group of features $c$, an ablation score can be written as

$$\Delta_c(x) = f(x) - f(x \setminus c), \tag{2.4}$$

where $x \setminus c$ denotes the input after removing $c$. A large positive value indicates that the removed information supported the original prediction, while a negative value indicates that its presence opposed it.

### 2.2.4 Explanation faithfulness

An explanation that appears reasonable to a human is not necessarily *faithful* to the model that produced the prediction. Faithfulness instead asks whether the evidence identified by an explanation actually influences the model output [43].

One way of testing this is through deletion-based evaluation. If an explanation identifies a set of evidence $E$ as important, that evidence can be removed and the model evaluated again. A substantial decrease in confidence,

$$f(x) - f(x \setminus E), \tag{2.5}$$

or a change in the predicted class provides evidence that the explanation identified information relevant to the original decision.

## 2.3 Adversarial Robustness

Machine-learning models can be vulnerable to adversarial examples, which are inputs deliberately modified to cause an incorrect prediction while preserving the properties relevant to the original task. For malware detection, this means modifying a malicious sample such that the classifier predicts it as benign, without removing its malicious functionality.

Given a malware sample $x$, a classifier $f$, and a set of allowed transformations $\mathcal{T}(x)$, an evasion attack can be expressed as

$$t^* = \arg\max_{t \in \mathcal{T}(x)} p_{\text{benign}}(t(x)), \tag{2.6}$$

where the transformations are constrained to preserve the semantics of the original sample.

Adversarial attacks are usually small perturbations directly in the model's feature space. For software, an arbitrary change to a feature vector does not necessarily correspond to a valid program. A problem-space attack instead modifies the underlying object while preserving its required functionality. Simply because a perturbation is possible in the chosen representation space, such as adding arbitrary API calls to a behavior report, does not mean that it is feasible in the real world. As such,

problem-space transformations imply additional real-world constraints that are not captured by theoretical feature-space perturbations [27] [5].

### 2.3.1 Adversarial Training

A common defense against adversarial attacks is adversarial training, in which adversarially modified samples are included during training. This can be viewed as a robust optimization problem [22]:

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[ \max_{t \in \mathcal{T}(x)} \mathcal{L}(f_\theta(t(x)), y) \right], \tag{2.7}$$

where the inner optimization searches for a transformation that maximizes the classification loss and the outer optimization updates the model to correctly classify these difficult examples.

However, applying adversarial training to malware must respect the problem-space constraints described previously. Previous works address this problem in the space of dynamic malware analysis by using reinforcement learning to search over a predefined set of feasible transformations, and then using the resulting adversarial examples to retrain the classifier to provide robustness [37].

Adversarial robustness has also been connected to explainability. Previous work has observed that robust models can produce more interpretable and stable feature attributions, potentially because adversarial training encourages smoother decision boundaries and reduces sensitivity to small perturbations [11, 40].

## 2.4 Related Work

Previous work on explainable malware detection has mostly relied on conventional feature-attribution methods. Card et al. train neural classifiers on dynamic and online malware-analysis features and use SHAP, LIME, and permutation importance to explain their predictions [6]. Such approaches can identify which input features influence a decision, but their output remains primarily a set of feature scores rather than a higher-level explanation of why the observed behavior is malicious.

The adversarial part of this thesis is most closely related to AutoRobust by Tsingenopoulos et al. [37]. AutoRobust constructs problem-space adversarial examples using reinforcement learning and uses them to iteratively retrain their baseline classifier to achieve robustness. This thesis starts from the same report dataset, behavioral setting and constraints, but explores a language model as the adversarial generator.

The final explanation architecture developed in this work is most closely related to Gspell by Baghershahi et al. [3]. Gspell addresses a different domain - the explanation of Graph Neural Network predictions - but follows a similar separation between prediction and explanation. Instead of asking the language model to reproduce the classifier, the trained GNN remains responsible for the prediction. Internal GNN representations are projected into the embedding space of an LLM

and inserted as soft prompts, allowing the LLM to generate a natural-language explanation conditioned on information from the predictive model.

The fusion mechanism used here is also related to architectures developed for multimodal language models. Flamingo connects a pretrained encoder to a frozen language model by first converting a variable number of encoder features into a small fixed set of tokens with a Perceiver Resampler, and then exposing these tokens to the language model through gated cross-attention [1]. Although Flamingo operates on visual features and was not designed for model explanation, the same architectural pattern provides a convenient way to expose a compact representation from another model to a language model.

These works cover the main components investigated in this thesis, but largely in isolation. Malware XAI work focuses on attribution methods, while AutoRobust studied adversarial robustness without producing natural language explanations, Gspell demonstrates that the internal representation of a dedicated predictor can be exposed to an LLM to produce such explanations, but considers graph classification rather than malware analysis. This thesis brings these directions together by studying the explainability and adversarial robustness of a behavioral malware classifier and coupling that classifier to a language model that generates a natural-language justification of its decision.

# Chapter 3

# Dataset and Baseline classifiers

## 3.1 Dataset

### 3.1.1 Dataset composition

This work is entirely based on the AutoRobust dataset released by Tsingenopoulos et al. together with their RAID 2024 paper *How to Train your Antivirus: RL-based Hardening through the Problem Space* [37], consisting of 26,258 CAPEv2 JSON reports for distinct binaries, labeled as either goodware or a specific family of malware.

The goodware binaries were obtained from software packages distributed through Chocolatey and cover the period from 2012 to 2020. The malware binaries were Portable Executable files from the Virlock, Vobfus, and Shipup families. These families were selected because they could exhibit malicious behavior during dynamic analysis without network connectivity. The malware samples cover the period from 2017 to 2020 [37].

The three malware families also have distinct behavioral profiles. Virlock is both ransomware and a polymorphic file infector, infecting files by replacing them with encrypted executables containing the malware body [10]. Vobfus is a worm family that spreads through removable and network drives, creates copies of itself and Autorun artifacts, and can download and execute additional malware [25]. Shipup is a Win32 trojan family, whose documented variants create and edit files, modify system configuration, and establish persistence through registry and Autorun changes [24].

The original binaries were executed by the AutoRobust authors in Windows 7 VMware virtual machines with 4 GB of RAM, four virtual CPUs, and a 128 GB virtual disk. The virtual machines had no network connectivity. CAPEv2 was used to orchestrate the analyses, and every binary was executed for 150 seconds. VMwareCloak was used and populated the virtual file system with common documents to reduce obvious sandbox artifacts [37]. This thesis starts from the resulting report summaries and does not repeat the binary-execution stage.

Table 3.1 and Figure 3.1 show the exact distribution found. Goodware samples are substantially underrepresented, accounting for only a third of the dataset, as goodware samples have proven much more difficult to procure [37].

Table 3.1: Composition of the local AutoRobust dataset used in this thesis.

| Class | Family | Reports | Share |
|---|---|---|---|
| Goodware | Goodware | 8,600 | 32.8% |
| Malware | Virlock | 9,142 | 34.8% |
| Malware | Vobfus | 5,383 | 20.5% |
| Malware | Shipup | 3,133 | 11.9% |
| Total | | 26,258 | 100.0% |

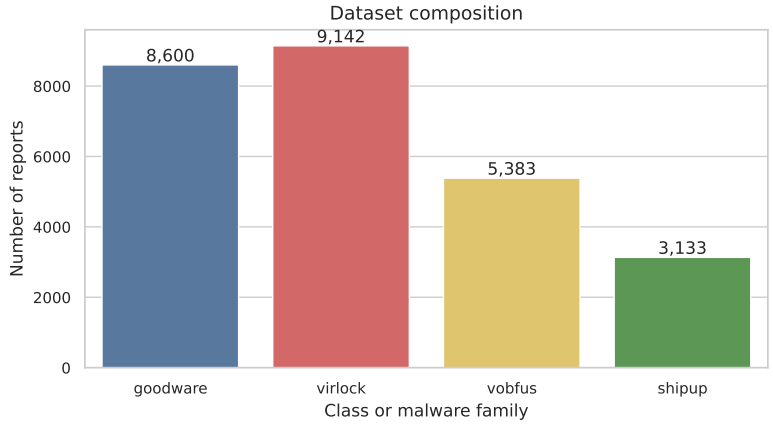



Figure 3.1: Number of reports in the local AutoRobust dataset, counted during the audit in this thesis.

A report does not contain the original executable. Instead, it records a summary of the interactions the binary has with the system at the system call granularity level. [37]. Every report uses the same high-level schema. The behavior categories are:

```
files, read_files, write_files, delete_files,
keys, read_keys, write_keys, delete_keys,
executed_commands, mutexes, created_services,
started_services, resolved_apis.
```

File categories contain the sets of observed, read, written, and deleted files, while the corresponding registry categories contain registry keys. The remaining categories contain executed commands, resolved external APIs, mutex names, and created or started service names. CAPEv2 records each distinct entry only once within a summary category [37].

The values stored in these categories are mostly strings. For example, a file entry can contain a complete Windows path, while a registry entry contains a registry hive and key. A report can therefore be treated as structured data, but it also contains a large amount of natural language-like text, allowing for the use of language models successfully in our task.

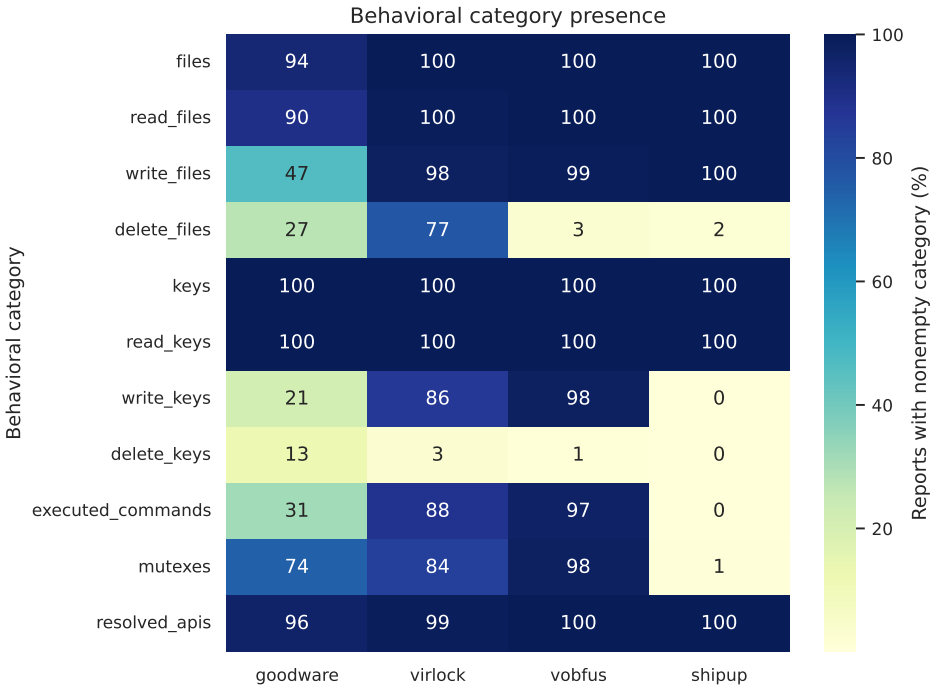



FIGURE 3.2: Presence of each behavioral category by class and malware family.

The categories are not equally common in all families: Figure 3.2 shows how often each category is present. Intuitively, some categories, such as registry reads and resolved APIs, occur in a large part of the dataset as basic components of any software. Other categories, especially service operations, are much less frequent, a useful fact for later explanation analysis as a rare category can have a large effect in a small number of reports while having little influence on average.

For example, Virlock shows a particularly high presence of file modification behavior because it is a type of ransomware with polymorphic file infection. It searches for suitable files and replaces them with new infected files containing both Virlock code and the encrypted original data. Reports show how the replacement process produces multiple file reads, writes, deletions. Registry keys are added to change security policies to help hide the malware or to disable security features. [7]

### 3.1.2 Report length limitations

Due to the chosen embedding model approach, we are limited by the maximum context length of the model used to convert the report into vector representations. `Qwen/Qwen3-Embedding-0.6B` was chosen for this purpose. This is an open-weight Qwen embedding model with a context length of 32,768 tokens, significantly higher than most models. This allows for us to embed the vast majority of our dataset in a single embedding without loss of information.

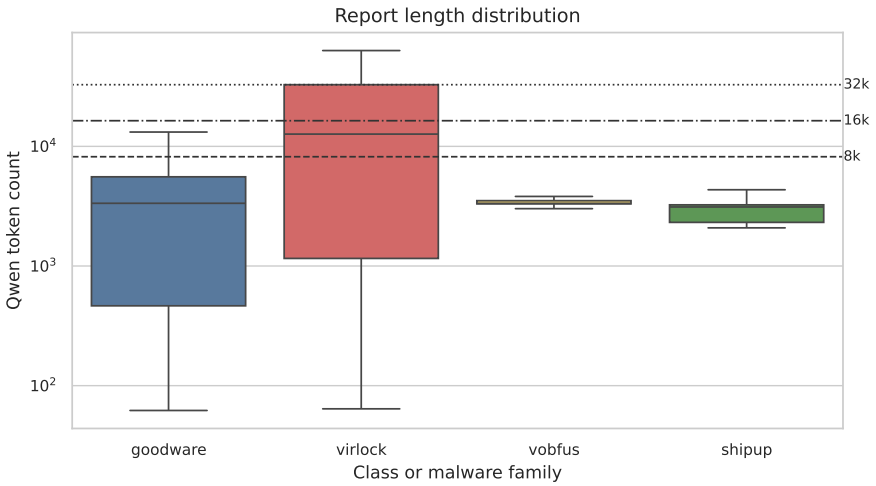



Figure 3.3: Token length distribution of the reports. The vertical marker shows the 32,768 token context limit.

Figure 3.3 shows that report length has a wide and skewed distribution. The median report in the final experimental collection contains 3,344 tokens, while the 95th percentile contains 23,517 tokens. Using only a short context such as 2,048 or 8,192 tokens would remove information from a substantial part of the selected data.

There were 3,582 unique reports above this context length limit. I chose to exclude these reports rather than truncate them. Truncation would remove an unknown part of the behavior log and could remove the activity that makes a sample malicious. This choice means that every report used by the classifiers is complete. The disadvantage is that the resulting experiment represents shorter reports better than very long reports.

A sliding-window approach could have been used to fully embed the entire dataset. However, while this approach would have allowed me to include the entire dataset for training our initial baseline classifiers, the context length limitation was further encountered during the process of LLM fine-tuning, where VRAM resource constraints led me to the same context length limitation during the training process.

### 3.1.3 Experimental split

For all the experiments attempted in this work, I partitioned the dataset into an experimental split. After removing duplicate copies, label conflicts, and over-context reports, 19,148 reports remained. Ten percent of the remaining malware reports were first reserved for later adversarial experiments. The other reports were divided into training, validation, and test sets using an 80/10/10 ratio.

The split was stratified by malware family and report-length bucket. Four length buckets were used: at most 2,048 tokens, 2,049–8,192 tokens, 8,193–16,384 tokens, and more than 16,384 tokens. This keeps both the family distribution and the

TABLE 3.2: Composition of the duplicate-safe experimental splits.

| Split | Goodware | Virlock | Vobfus | Shipup | Total |
|---|---|---|---|---|---|
| Training | 4,918 | 3,287 | 3,836 | 2,237 | 14,278 |
| Validation | 615 | 411 | 479 | 280 | 1,785 |
| Test | 615 | 411 | 480 | 279 | 1,785 |
| Adversarial | 0 | 456 | 533 | 311 | 1,300 |
| Total | 6,148 | 4,565 | 5,328 | 3,107 | 19,148 |

length distribution similar between the three classification splits. Table 3.2 shows the resulting splits and their distribution in detail.

## 3.2 Baseline Classifiers

### 3.2.1 Models and validation procedure

The first experiment attempted consisted of training multiple baseline ML classifiers for this task. I chose multiple classic approaches for text classification:

- Majority classifier - always predicts the most common training class; simple lower bound.
- Logistic regression tests how well the classes can be separated by a linear decision boundary.
- Decision trees provide a simple nonlinear classification model, easily explainable.
- XGBoost provides a stronger performance-wise ensemble of decision trees.
- Multilayer perceptron - provides a neural classifier that can later be connected to a language model.

Hyperparameters were selected using validation macro-F1, with validation accuracy used to break a tie. The test labels were not used during model or threshold selection. Table 3.3 lists the selected configurations.

The selected MLP reached its best validation result at epoch 25, with 99.50% accuracy and 0.9944 macro-F1. Training stopped after epoch 31 and the parameters from epoch 25 were restored. Figure 3.4 shows the training history.

### 3.2.2 Evaluation

The primary measure is macro-F1. It calculates F1 separately for goodware and malware and then gives both classes equal weight. I also report accuracy, balanced accuracy, malware precision, malware recall, and ROC-AUC.

Table 3.3: Hyperparameters selected on the validation set.

| Model | Selected configuration |
|---|---|
| Majority | `strategy=most_frequent`; threshold = 0.95 |
| Logistic regression | `C=10`; `solver=lbfgs`; threshold = 0.51 |
| Decision tree | `max_depth=24`; `min_samples_leaf=2`; threshold = 0.95 |
| XGBoost | `n_estimators=700`; `max_depth=4`; `learning_rate=0.05`; `min_child_weight=1`; `subsample=0.85`; `colsample_bytree=0.85`; threshold = 0.795 |
| MLP | `hidden_layer_sizes=(512, 256)`; `dropout=0.30`; `batch_size=64`; `learning_rate_init=`$10^{-3}$; threshold = 0.58 |

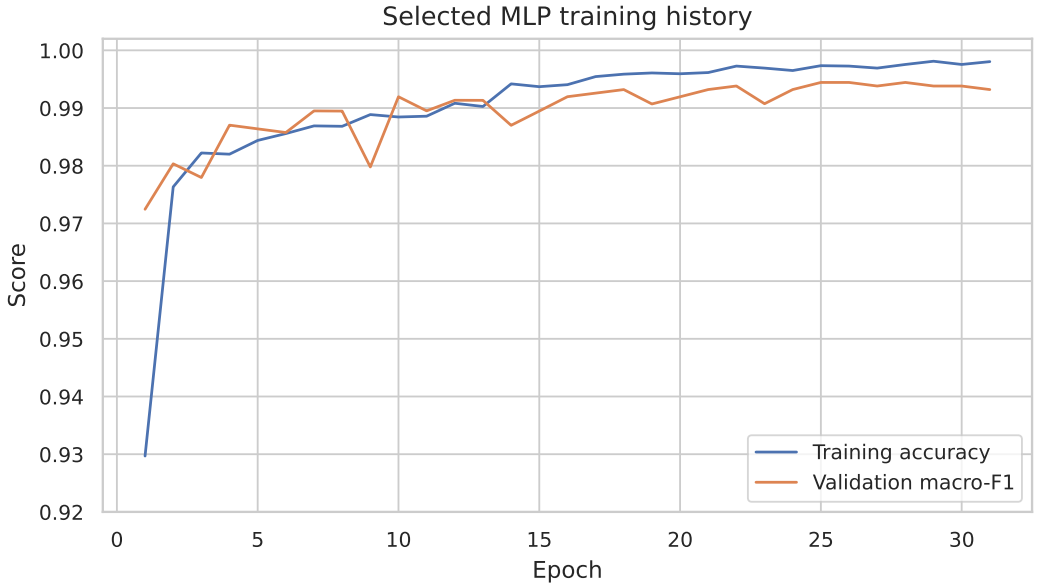



Figure 3.4: Training history of the selected MLP.

Table 3.4 gives the final results on the duplicate-safe test set. The zero-shot LLM was evaluated later on the smaller common explanation subset reported in Chapter 8, so its results are not directly comparable and are not included in this table.

The MLP obtained the highest test macro-F1, 0.9864, and the highest accuracy, 98.77%. Its 95% bootstrap interval was 98.29–99.27% for accuracy and 0.9811–0.9919 for macro-F1. The confusion matrix contains 604 correctly classified goodware reports, 11 false positives, 11 false negatives, and 1,159 correctly classified malware reports.

Figure 3.5 summarizes macro-F1 and the MLP confusion matrix. The confidence intervals in the performance plot also show that the two strongest models are very close.

TABLE 3.4: Performance on the duplicate-safe test set.

| Model | Accuracy | Balanced acc. | Macro-F1 | Malware prec. | Malware recall | ROC-AUC |
|---|---|---|---|---|---|---|
| Majority class | 0.655 | 0.500 | 0.396 | 0.655 | 1.000 | 0.500 |
| Logistic regression | 0.980 | 0.980 | 0.978 | 0.990 | 0.980 | 0.997 |
| Decision tree | 0.952 | 0.947 | 0.947 | 0.965 | 0.962 | 0.924 |
| XGBoost | 0.987 | 0.988 | 0.985 | 0.997 | 0.983 | 0.999 |
| MLP | **0.988** | 0.986 | **0.986** | 0.991 | **0.991** | 0.999 |

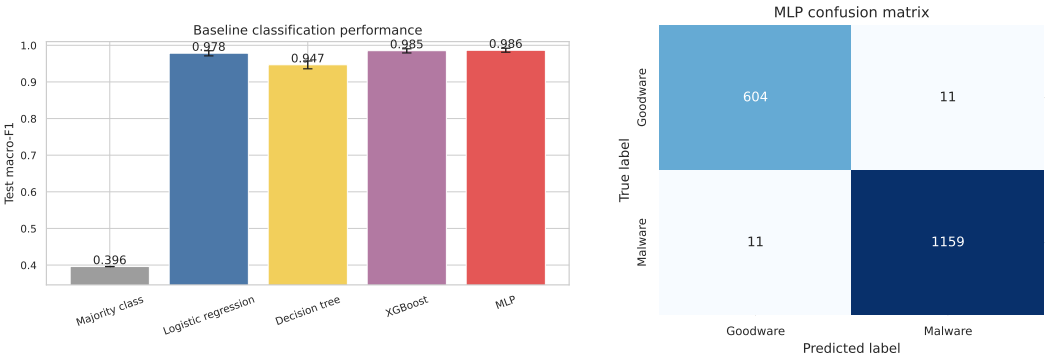



FIGURE 3.5: Test macro-F1 with bootstrap intervals (left) and the MLP confusion matrix (right).

TABLE 3.5: Accuracy by class and malware family.

| Model | Goodware | Virlock | Vobfus | Shipup |
|---|---|---|---|---|
| Logistic regression | 0.980 | 0.946 | 0.998 | 1.000 |
| Decision tree | 0.933 | 0.917 | 0.979 | 0.996 |
| XGBoost | 0.993 | 0.959 | 0.994 | 1.000 |
| MLP | 0.982 | **0.976** | **0.998** | 1.000 |

### 3.2.3 Performance by malware family

An overall malware score can hide a weak malware family. I therefore measured accuracy separately for goodware, Virlock, Vobfus, and Shipup. The results are shown in Table 3.5 and Figure 3.6.

Virlock was the most difficult malware family for every learned classifier. The MLP classified 97.6% of Virlock reports correctly, compared with 99.8% for Vobfus and 100% for Shipup. XGBoost performed better on goodware but worse on Virlock. This explains why the two models have similar overall accuracy while making different errors.

### 3.2.4 Experimental environment

I must note that the experiments in this thesis were run on a virtual machine equipped with two NVIDIA GeForce RTX 5090 GPUs. The available GPU mem-

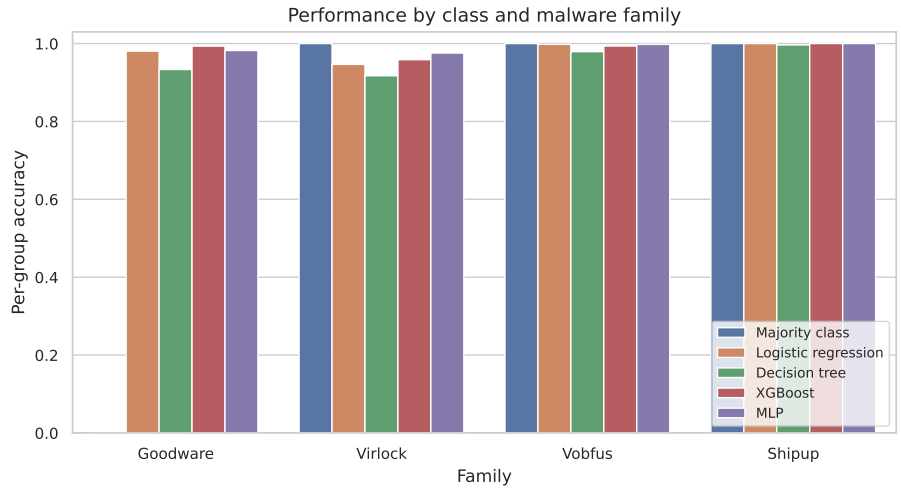



Figure 3.6: Test accuracy for goodware and each malware family.

ory and compute budget influenced several implementation choices used throughout the thesis, including quantized language models, LoRA fine-tuning, gradient checkpointing, and small per-device batch sizes. While training the baseline classifiers on such hardware was not challenging, the VRAM and computational power restrained future decisions regarding model sizes, context lengths or number of samples evaluated.

# Chapter 4

# Robustness and Explainability

## 4.1 Explainability Analysis

While we have easily produced a model capable of discerning between malware and goodware logs, its probability alone does not show why a certain label is chosen. For an end user analyst, this is an obvious practical limitation, as they must be able to inspect the evidence behind a certain result and determine whether the model relied on meaningful behavior or an accidental correlation in the dataset.

Explainability methods attempt to expose which parts of an input contributed to a model decision. In this chapter, I use three complementary approaches: category ablation for a coarse behavior-level view, Kernel SHAP for local entry-level attribution, and Integrated Gradients for attribution through the complete differentiable pipeline. All experiments in this chapter are performed on the original MLP selected in Chapter 3.

For the selected MLP, the main difficulty is the representation used by the classifier. A JSON behavior report $x$ is first processed by the embedding model $E$, which produces a 1024-dimensional vector, taken by the MLP $g$ as input to produce the malware probability:

$$f(x) = g(E(x)).$$

As such, the MLP has no direct knowledge of named features such as `write_files`, `executed_commands`, or individual registry keys, but of embedding coordinates composed from the entire report. Naive direct attribution on the 1024 MLP inputs would explain the classifier mathematically, but such an output would be useless for an analyst interested in how a certain label was selected.

For these reasons we use multiple industry-standard XAI approaches to project the explanations back onto the original report to understand our current baseline classifier [21, 35].

Table 4.1: Explainability settings.

| Method | Configuration |
|---|---|
| Category ablation | 32 correctly classified test reports, eight from each cohort; 227 supported report–category combinations. |
| Kernel SHAP | One 730-token Virlock report; at most 20 JSON entries; zero-entry background; 128 sampled coalitions. |
| Integrated Gradients | One 598-token Virlock report; padding-token baseline; 16 integration steps. |

### 4.1.1 Category ablation

Category ablation is the most direct behavior-level explanation: for each selected report, compute the original malware probability, then remove all entries from one behavior category, serialize and embed the modified report, and then recompute its probability. For a report $x$ and category $c$, signed change is computed as:

$$\Delta_c(x) = f(x) - f(x_{\setminus c}).$$

where $x_{\setminus c}$ is the report after category $c$ has been cleared. A positive value means that the removed category supported the malware prediction. A negative value means that its presence reduced the malware score. I used $|\Delta_c(x)|$ to compare the overall sensitivity to different categories. Sensitivity was computed over 32 correctly labeled test reports, distributed equally over the classes. This experiment analyzed in total 227 report-category combinations, available in Table 4.1.

Table 4.2 and Figure 4.1 show that the largest mean absolute changes were obtained after removing `resolved_apis` and `files`, of 0.0607 and 0.0284, respectively. At first sight, this could suggest that these categories generally dominate the classifier. But the very small mean changes, while their maximum changes were of 0.9954 and 0.8054, show a very unequal distribution in fact. As such, the averages were driven by a small number of highly sensitive reports.

This effect is especially clear for Virlock, due to its ransomware file-intensive nature, but it does not show that these categories are always malicious or globally important. Instead, it shows a sparse and family-dependent reliance: a category can have almost no effect in many reports and become decisive in a few. [7]

Category ablation has the advantage of using understandable units. It is easy to state that file activity or resolved APIs affect a prediction in a certain way. However, it is a very broad unit, as removing a certain category deletes hundreds of benign and suspicious entries at the same time. It also creates an out-of-distribution problem, as this approach creates reports that would not occur in a real sandbox (programs that do not touch any files or APIs). The experiment therefore measures model sensitivity to this intervention, not a causal malware mechanism. [39]

TABLE 4.2: Category-ablation statistics.

| Category | Reports | Mean | Median | Maximum |
|---|---|---|---|---|
| resolved_apis | 32 | 0.060697 | 0.000005 | 0.995366 |
| files | 30 | 0.028383 | 0.000048 | 0.805430 |
| read_keys | 32 | 0.000341 | 0.000000 | 0.009422 |
| keys | 32 | 0.000138 | 0.000000 | 0.002139 |
| read_files | 30 | 0.000068 | 0.000000 | 0.001184 |
| executed_commands | 16 | 0.000027 | 0.000001 | 0.000178 |
| mutexes | 16 | 0.000017 | 0.000000 | 0.000179 |
| delete_files | 1 | 0.000001 | 0.000001 | 0.000001 |
| write_keys | 13 | 0.000001 | 0.000000 | 0.000006 |
| write_files | 25 | 0.000001 | 0.000000 | 0.000009 |

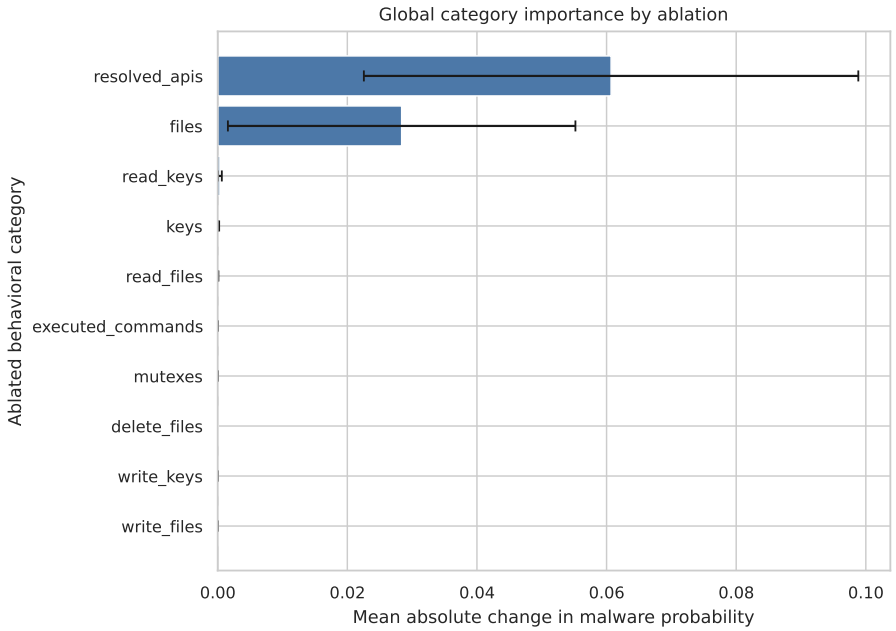



FIGURE 4.1: Category-ablation effects.

Table 4.3: Largest local SHAP contributions.

| Category | Entry | SHAP value |
|---|---|---|
| files | C:\Windows\System32\tzres.dll | 0.4827 |
| files | C:\Windows\System32\Kbfpig32.exe | -0.1504 |
| keys | HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\OLEAUT | -0.1392 |
| read_keys | HKEY_LOCAL_MACHINE\SOFTWARE\MICROSOFT\WINDOWSNT\CURRENTVERSION\GRE_Initialize\DisableMetaFiles | -0.1004 |
| read_files | C:\Windows\System32\tzres.dll | 0.1001 |
| keys | HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Microsoft\Windows\WindowsErrorReporting\WMR\Disable | -0.0957 |

### 4.1.2 SHAP

Kernel SHAP was used to obtain a more detailed local explanation. The explained sample was a correctly classified Virlock report containing 730 tokens. Exact Shapley computation grows exponentially with the number of features [21], so it was infeasible for a report containing many entries.

I therefore selected at most 20 individual behavior entries from the report, where an entry is one observed value such as a file path, registry key, executed command, or resolved API. Each entry belongs to one of the behavior categories introduced in Section 3.1.

For Kernel SHAP, each selected entry is represented by a binary feature indicating whether it is retained in a modified version of the report. A coalition is one subset of these entries. Kernel SHAP sampled 128 such coalitions; for each one, the corresponding JSON report was reconstructed, embedded again, and evaluated by the MLP.

It is visible in Figure 4.2 and Table 4.3 that the strongest positive local contribution can be attributed to `C:\Windows\System32\tzres.dll` from the `files` category, with a SHAP value of approximately +0.483. Also, the same path in `read_files` received approximately +0.100, given the context aware nature of the model.

Negative values were assigned to `C:\Windows\System32\Kbfpig32.exe`, the `Microsoft\OLEAUT` registry key, a `GRE_Initialize\DisableMetaFiles` entry, and a Windows Error Reporting `WMR\Disable` key.

The sign of a local SHAP value must be read relative to this report and the selected background [21, 34]. A negative value does not mean that an entry is always benign, and a positive value does not mean that it is inherently malicious. It only describes how the selected entry affected this prediction under the sampled coalitions.

To check whether the attributed entries represented clear malware evidence, I also measured their prevalence in malware and goodware reports. Table 4.4 shows that `Microsoft\OLEAUT` occurred in 62.205% of malware reports and 59.597% of goodware reports. `GRE_Initialize\DisableMetaFiles` occurred in 43.145% of

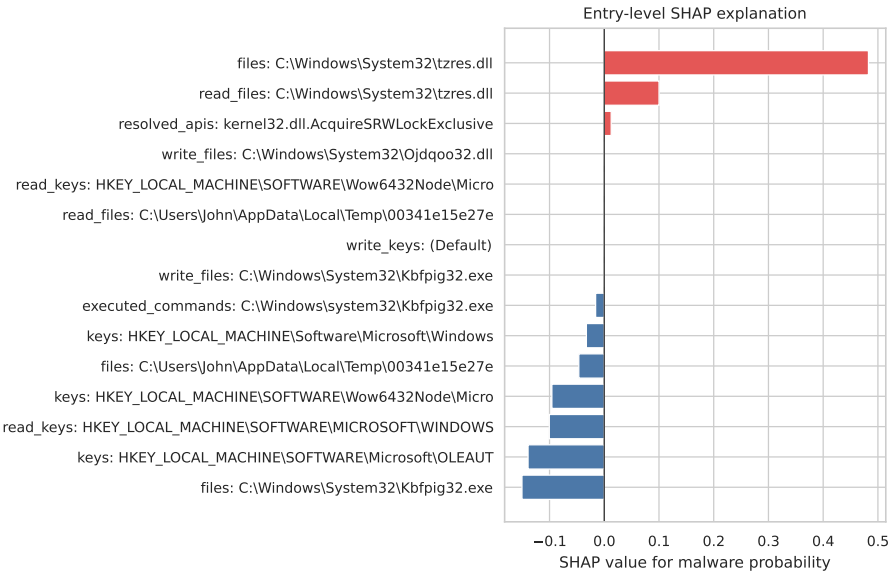



FIGURE 4.2: Local SHAP explanation.

TABLE 4.4: Prevalence of SHAP-selected entries.

| Entry | Malware | Goodware |
|---|---|---|
| `Microsoft\OLEAUT` | 62.205% | 59.597% |
| `GRE_Initialize\DisableMetaFiles` | 43.145% | 56.327% |
| Windows Error Reporting `WMR` | 95.291% | 67.437% |
| `C:\Windows\System32\tzres.dll` | 4.966% | 0.000% |

malware and 56.327% of goodware. The Windows Error Reporting `WMR` entry occurred in approximately 95.291% of malware and 67.437% of goodware. The `tzres.dll` path is a normal Windows component, but it occurred in 4.966% of malware reports and none of the goodware reports in this dataset.

These observations show that local contribution, global frequency, and semantic maliciousness are different properties. A feature can be useful to the classifier because it is correlated with a malware family, the sandbox environment, or the data-collection process, even when it is not malicious by itself. SHAP reports a local contribution made by the model; it does not determine whether the contribution is a valid security reason.

The result suggests possible reliance on dataset-specific shortcuts, but it does not prove that the classifier relies only on shortcuts. The SHAP analysis contains one detailed local example, and the selected entries interact with other behaviors in the report. A broader stability and coverage experiment would be required before making a global claim. Local perturbation explanations can be unstable and should be checked through repeated sampling or uncertainty estimates [32].

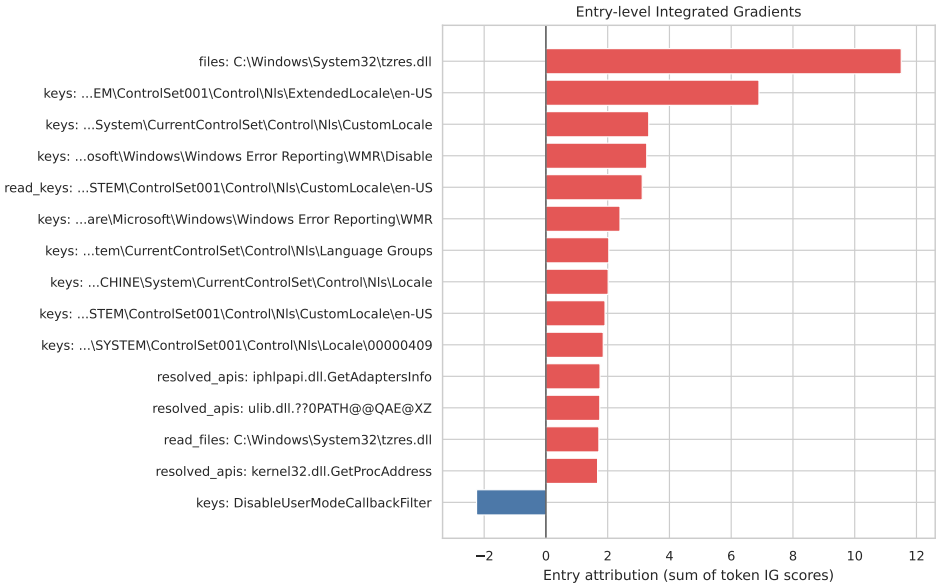



Figure 4.3: Entry-level Integrated Gradients.

### 4.1.3 Integrated Gradients

Integrated Gradients [35] was applied to a second correctly classified Virlock report. The input contained 598 tokens, of which 597 non-special tokens were included in the explanation. Attributions were computed through the complete differentiable pipeline at the Qwen input embedding layer. A sequence of padding tokens was used as the reference, the malware output was selected as the target, and the path integral was approximated using 16 steps.

To make the Integrated Gradients output more interpretable, I aggregated the attributions of all tokens belonging to the same behavioral entry. Figure 4.3 shows the resulting entry-level attribution. Positive values support the malware prediction, while negative values reduce it.

The strongest positive attribution is assigned to `C:\Windows\System32\tzres.dll`, followed mostly by registry entries related to Windows locale and error-reporting configuration. In contrast, the registry key `DisableUserModeCallbackFilter` receives a negative attribution.

However, Integrated Gradients depends on the chosen reference input. Here, a sequence of padding tokens was used as the baseline. While convenient, this does not correspond to a semantically neutral behavior report, and the resulting attributions should therefore be interpreted relative to this reference rather than as absolute measures of feature importance [35].

### 4.1.4 Limitations of analyzed approaches

The three explainability methods provide complementary but incomplete views of the classifier. Category ablation produces understandable category-level effects, but it is too coarse to identify which individual behaviors caused the change and may create unrealistic reports by removing an entire class of activity. Kernel SHAP provides signed contributions for individual JSON entries, but it is computationally expensive, depends on the selected background and sampled coalitions, and becomes impractical for reports containing thousands of entries. Integrated Gradients follows the complete differentiable pipeline, but its token-level output is fragmented and strongly dependent on the chosen baseline.

More importantly, the obtained attributions suggest that the MLP may partially rely on dataset-specific shortcuts rather than explicitly malicious behavior. In both the Kernel SHAP and IG examples, the strongest positive contribution was assigned to the normal Windows component `C:\Windows\System32\tzres.dll`, a Windows resource DLL used for localized time-zone names [26]. This path occurred in several malware reports but in none of the goodware reports in the dataset, making it predictive even though its presence is not inherently malicious. Other highly attributed entries, `Microsoft\OLEAUT`, `GRE_Initialize\DisableMetaFiles`, and Windows Error Reporting registry entries, were also common operating-system artifacts rather than direct evidence of malicious activity.

These observations do not prove that the classifier relies exclusively on shortcuts. The SHAP and Integrated Gradients experiments are local analyses, and the attributed features interact with the rest of each report. Nevertheless, they show why attribution scores alone are insufficient as analyst-facing explanations. A ranked list can reveal which paths or tokens affected the model, but it cannot determine whether those features are semantically malicious, whether they are normal operating-system artifacts, or whether the model has learned an accidental dataset correlation.

The scale and structure of the reports further limit these approaches. Exact Shapley computation is infeasible for reports containing thousands of entries, while sampled Kernel SHAP requires a new embedding pass for every coalition [21]. Arbitrary entry combinations and complete category deletion may also fall outside the distribution of realistic sandbox reports [39]. Correlated behaviors make it difficult to assign an isolated effect to one entry, and tokenization divides complete behaviors into fragments that are difficult for an analyst to interpret.

These limitations motivated the later use of a language model to produce structured, analyst-readable explanations. The language model does not replace or correct the MLP: if the classifier relies on a shortcut, a generated explanation may still rationalize that shortcut. Its role is instead to organize the observed suspicious behaviors, benign behaviors, and missing evidence into natural language, while the MLP remains responsible for classification.

## 4.2 Robustness as a Working Hypothesis

The explainability analysis raises a practical concern: some features may be predictive only because they are correlated with a particular malware family, sandbox configuration, or data-collection process. A classifier that relies on such shortcuts can achieve high test accuracy while being vulnerable to very simple attacks such as the malware executable using a different filename.

This motivates the use of robustness as a diagnostic for explanation quality. A feature is more credible as evidence when the classifier continues to rely on it under small, valid variations of the input. Conversely, if a semantically irrelevant modification causes a large change in the prediction or attribution map, the original explanation may reflect a fragile correlation rather than a stable decision rule.

Adversarial training provides the mechanism used to pursue this robustness. Its standard min-max formulation searches for inputs that maximize the model loss and then updates the model to minimize the loss on those difficult examples [22].

The reason to expect this effect lies in what adversarial training penalizes. Standard training has no reason to distinguish durable behavioral evidence from a brittle feature that happens to predict the label in the training distribution. Such features can be highly predictive while remaining sensitive to small changes and semantically meaningless to a human observer [15]. In the present dataset, filenames, paths, mutex names, and registry values may act as such features. If replacing one of these artifacts repeatedly causes a large decrease in malware probability, the classifier is relying on information that can be changed without removing the malicious behavior represented by the report.

Training on these transformed reports with their original malware labels makes this reliance costly. To classify both the original report and its valid variants consistently, the model is encouraged to reduce its sensitivity to the modified artifacts and rely more strongly on evidence that persists across the allowed transformations. Robustness is therefore relevant here not as a general property of the classifier, but specifically as invariance along directions that should not determine the malware label.

This invariance may also improve the resulting feature attributions. When the decision boundary is close or highly irregular, a small and semantically irrelevant modification can change its local orientation, causing the gradient and the attributed features to change substantially. Increasing the margin around an input and smoothing the local decision boundary can reduce this sensitivity and can cause input gradients to better approximate the directions that locally separate the predicted class from its alternatives [11, 40].

As such, hardening against valid behavioral transformations may produce explanations that are less dominated by brittle artifacts and more consistent across semantically equivalent reports. Chapters 5 and 6 test this hypothesis by developing adversarial attacks against the MLP and subsequently using the generated examples to harden it. Chapter 8 then compares the explanations generated with the baseline model against those generated with the robustified model.

# Chapter 5

# LLM-Based Adversarial Evaluation

## 5.1 Introduction

The goal of this chapter is to build an automated adversarial attacker for the MLP classifier. Before improving the robustness of the classifier, I first needed a way to find weaknesses in its decision boundary.

The closest previous approach is AutoRobust, which uses reinforcement learning to generate feasible adversarial changes for the same behavioral dataset [37]. In this project, I follow its attack model, but use a language model to choose the changes. This allows the attacker to use the meaning and structure of the behavior report instead of selecting only from a fixed set of actions. Language-model-based adversaries have achieved high attack success rates in related text-classification settings [44].

The LLM attacker receives a malware behavior report and proposes changes that make the report look more similar to goodware. The modified report is embedded again with `qwen3-embedding:0.6b` and passed through the same MLP, searching for weaknesses in its decision boundary.

I trained the evaluator in two stages. Supervised fine-tuning (SFT) first taught an 8-billion-parameter Qwen3 model to produce valid changes. Group Relative Policy Optimization (GRPO) then used the MLP probability as direct feedback. Qwen3 separates its reasoning trace from its final answer. In this task, the reasoning block gives the model space to plan which behavior categories to modify and to check the attack constraints before producing the final JSON diff [41, 29].

## 5.2 Task and Threat Model

Let $x$ denote a malware behavior report. The embedding model $E$ converts the serialized report into a 1024-dimensional vector, and the MLP $f$ produces a probability distribution over goodware and malware:

Table 5.1: Permitted AutoRobust transformations [37].

| Behavior category | Add | Replace | Remove |
|---|---|---|---|
| File categories | Yes | Yes | No |
| Registry-key categories | Yes | Yes | No |
| Executed commands | Yes | Yes | No |
| Resolved APIs | Yes | No | No |
| Mutexes | Yes | Yes | No |
| Created and started services | Yes | Yes | No |

$$p(x) = f(E(x)). \tag{5.1}$$

The adversarial model generates a JSON difference, denoted by $\delta$. A deterministic function $A$ applies this difference to the report:

$$x' = A(x, \delta). \tag{5.2}$$

The implemented evaluator counts an attack as successful when the modified report is classified as goodware:

$$p_{\text{goodware}}(x') > 0.5. \tag{5.3}$$

The attack model follows AutoRobust because that work uses the same dataset and behavior categories. An attack should contain "all those and only those transformations that are feasible, threatening, and can actually be carried out through the problem-space" [37]. Therefore, additions are allowed in every category, while replacements are allowed in every category except `resolved_apis`, because replacing an API call can break the original program. Removals are not allowed, since removing an observed behavior would generally require the executable to be rewritten or refactored [37]. These restrictions are summarized in Table 5.1.

The LLM expresses these transformations as `add` and `edit` operations. Every `old` value in an edit must exactly match an existing entry. A shortened output has the following form:

```
{
  "add": {
    "read_files": [
      "C:\\Windows\\Fonts\\staticcache.dat"
    ]
  },
  "edit": {
    "executed_commands": [
      {
        "old": "suspicious.exe",
        "new": "update_service.exe"
      }
    ]
  }
}
```

The prompt also contains entries sampled from goodware reports and the earlier ranking of important behavior categories. The chosen attack model separates an action into what operation to perform, where to perform it, and how to construct the new value [37]. The LLM makes the same decisions jointly when it generates the JSON diff. It receives a grey-box view of the classifier: it cannot inspect the MLP weights, but it knows which behavior groups are influential and receives the final classifier score.

## 5.3 Initial Adversarial Data

Before fine-tuning the smaller model, I needed examples of successful attacks. I generated them with the larger `Qwen3.5:35b` reasoning model. The model received the complete malware report, the required JSON schema, one worked example, and the earlier category-importance ranking. I also gave it examples of common goodware behavior and vocabulary released with AutoRobust [37]. The main part of the generation prompt is shown in Listing 5.1; the category ranking and goodware examples were inserted into the two marked fields for each request.

Listing 5.1: Abbreviated system prompt used to generate the initial attacks.

```
You are a cybersecurity expert specializing in malware evasion analysis.
Your task is to generate an adversarial perturbation (a "diff" in JSON
format) that, when applied to a malware behavioral log, confuses a
classifier into thinking it is "goodware".

You have TWO types of operations available:
1. add: Insert new entries characteristic of legitimate software behavior.
2. edit: Replace existing suspicious entries with benign-looking ones.
You are not allowed to edit resolved_apis, but you can add new API entries.

Rules:
1. Output only the JSON diff and nothing else.
2. Use the required {"add": {...}, "edit": {...}} schema.
```

```
3. Use only the valid behavior categories.
4. Every "old" edit value must exactly match an entry in the original log.
5. Replacement entries must look like legitimate software behavior.
6. Added entries must be realistic and must not duplicate existing entries.
7. Do not remove entries; only add or replace them.
8. Focus on the most important behavior categories.

Category importance:
<category ranking inserted here>

Common goodware behavioral entries:
<AutoRobust reference data inserted here>
```

For every generated diff, I applied the changes and evaluated the modified report with the MLP, and only successful attacks were retained. Using this method on the separately reserved adversarial training split, I could obtain only 152 successful adversarial examples. This is a small dataset, but the objective of this stage was narrow: teaching a pretrained model the attack instructions, reasoning pattern, and output schema. Previous works such as LIMA similarly showed that a strong pretrained language model can learn specific response formats from limited, high-quality supervision; one of its structure experiments found that even six examples could determine whether the model produced a complex output format [47]. The examples were used as a warm start rather than as the only source of task feedback, and the GRPO stage optimized the model directly against the classifier.

## 5.4 Supervised Fine-Tuning

The student model was Qwen3-8B loaded in 4-bit form. I used Low-Rank Adaptation (LoRA) on the attention and feed-forward projections. LoRA keeps the original weights fixed and learns smaller low-rank updates, which reduces the amount of trainable memory [14].

Each training example contained the instruction, the original report, the teacher reasoning, and the successful JSON diff. The maximum sequence length was 32,000 tokens. Samples that did not fit were not included. Training ran for 600 update steps, using batches of one report and two gradient accumulation steps.

SFT taught the model the expected response structure and the strategies found by the larger teacher. This stage was necessary because experiments with GRPO alone initially had to explore both the attack and the output format. Starting from an SFT model made parseable candidates much more common and gave the reinforcement stage a useful starting point.

## 5.5 GRPO

SFT teaches the student to imitate successful examples, but it does not show whether a new diff improves the classifier score. I therefore continued training with GRPO. The objective had two forms of alignment. First, the model had to remain aligned

TABLE 5.2: SFT configuration.

| Parameter | Value |
|---|---|
| Base model | Qwen3-8B, 4-bit |
| Training examples | 152 |
| Maximum sequence length | 32,000 tokens |
| LoRA rank / scaling | 16 / 32 |
| Batch size / accumulation | 1 / 2 |
| Learning rate | $5 \times 10^{-5}$ |
| Optimizer | 8-bit AdamW |
| Warm-up steps | 30 |
| Update steps | 600 |

with the response protocol: it had to preserve the thinking block and produce JSON that the evaluator could apply. Second, it had to align its attack with the actual weakness of the MLP by increasing the classifier's goodware probability.

For each prompt, GRPO sampled four completions and compared their rewards. The relative reward inside this group was then used to update the policy [29]. The nominal reward implemented for one completion was:

$$R = r_{\text{json}} + r_{\text{structure}} + 0.5r_{\text{think}} + 4r_{\text{evasion}}. \tag{5.4}$$

The JSON-validity reward, $r_{\text{json}}$, was $+1$ when the parser could extract a JSON diff and $-1$ otherwise. This component was necessary because the classifier reward cannot be computed for an output that cannot be parsed. However, a valid JSON is not enough. The structure reward, $r_{\text{structure}}$, checks that the JSON follows the previously defined schema. A fully valid diff received $+1$, otherwise invalid structures receive $-1$.

The thinking reward, $r_{\text{think}}$, was $+1$ when the output contained a non-empty `<think>...</think>` block of at least 20 characters and $-0.5$ otherwise. It preserved the response structure learned during SFT, but it only measured the presence and minimum length of the reasoning block, not whether the reasoning was correct.

The main task reward used the goodware probability $p_g$ produced after the diff was applied:

$$r_{\text{evasion}}(p_g) = \begin{cases} 2 + p_g, & p_g > 0.5, \\ 2p_g, & 0.1 < p_g \leq 0.5, \\ p_g - 0.5, & p_g \leq 0.1. \end{cases} \tag{5.5}$$

A candidate below 0.1 received a negative reward, since it left the classifier strongly on the malware side. Between 0.1 and 0.5, the reward increased continuously with the goodware probability. This gave the model useful feedback even when an early candidate did not yet evade the classifier.

Once $p_g$ passed 0.5, the added constant produced a clear jump for a successful attack. The model was therefore encouraged to preserve a usable output format, but not at the expense of the actual evasion objective.

Table 5.3: GRPO configuration.

| Parameter | Value |
|---|---|
| Initialization | SFT model |
| Training prompts | 200 |
| Epochs / update steps | 10 / 2,000 |
| Generations per prompt | 4 |
| Temperature | 0.9 |
| Prompt / completion limit | 16,000 / 16,000 tokens |
| Batch size / accumulation | 1 / 4 |
| Learning rate | $5 \times 10^{-6}$ |
| LoRA rank / scaling | 16 / 32 |
| KL coefficient | 0.01 |

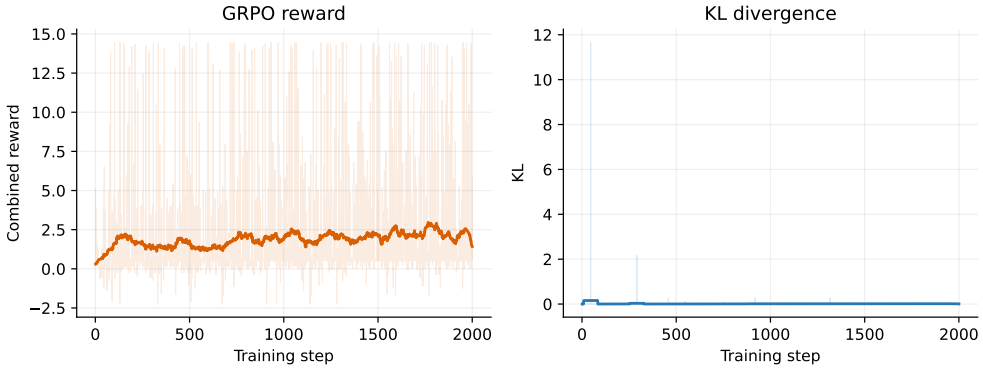



Figure 5.1: GRPO reward and KL divergence.

GRPO used 200 sampled malware prompts for 10 epochs, resulting in 2,000 update steps. Table 5.3 lists the complete training configuration.

Figure 5.1 shows the combined reward and KL divergence recorded during the original run. The reward is noisy because each step contains new reports and sampled completions. The moving average is therefore more useful than any individual point. The KL values remain small for most of the run, showing that the adapter did not move arbitrarily far from the SFT policy.

## 5.6 Evaluation & results

The two obtained models, trained with both SFT+GRPO and just SFT, were evaluated on their evasion rate. Each evaluation run was done on a sampled batch of 400 reports, due to the cost of their inference. For every report, the model generated four completions at temperature 0.9, with a completion budget of 16,000 tokens. Every parseable diff was applied to the report and rescored. A report was considered evaded if at least one of its four candidates produced $p_{\text{goodware}} > 0.5$.

The SFT model produced an evasion rate of 38.62%, while the GRPO model

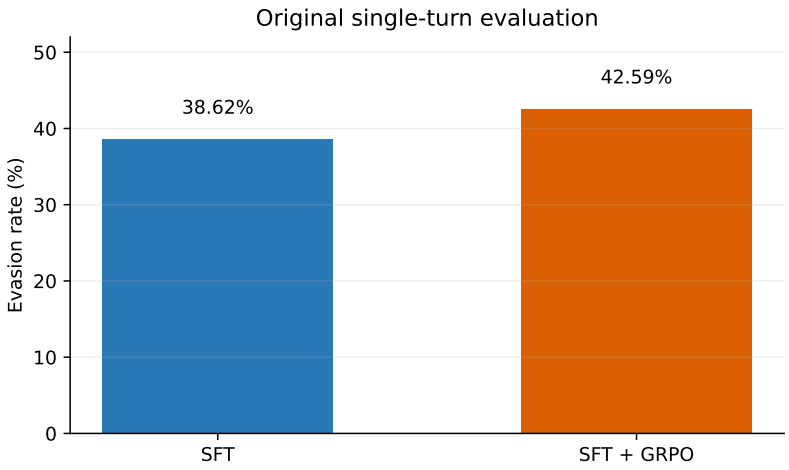



FIGURE 5.2: Evasion rates.

produced an evasion rate of 42.59%. Both models produced at least one parseable completion for every evaluated report. However, this was considered a poor result for an evaluator intended to expose weaknesses in the classifier. Even with four attempts per report, more than half of the samples remained classified as malware, while classifier-guided GRPO improved the evasion rate by only about four percentage points

The result shows a modest improvement after classifier-guided training. The SFT model already achieved a substantial evasion rate because it learned directly from successful teacher examples. GRPO improved this result further by optimizing against the MLP score.

The score distribution gives more information than the binary threshold. The mean best goodware probability was 0.366 for SFT and 0.412 for GRPO. The medians were 0.046 and 0.116 respectively. As shown in Figure 5.3, the GRPO curve is shifted toward larger goodware scores. This means that GRPO also improved several unsuccessful candidates, not only those that crossed 0.5.

## 5.7 Generated Changes

Both models relied most strongly on file reads, registry activity, and executed commands. These categories contain many common Windows paths and settings, so they provide a large pool of plausible benign-looking behavior. The exact report categories are shown in Table 5.4. The pattern also agrees with the earlier observation that the classifier is sensitive to file and registry activity.

The SFT model produced approximately 9.97 effective additions per successful attack, with a maximum of 21. The GRPO model produced approximately 10.41 per successful attack, with a maximum of 17. The improvement therefore did not come from a dramatically larger perturbation. The two models used a similar average

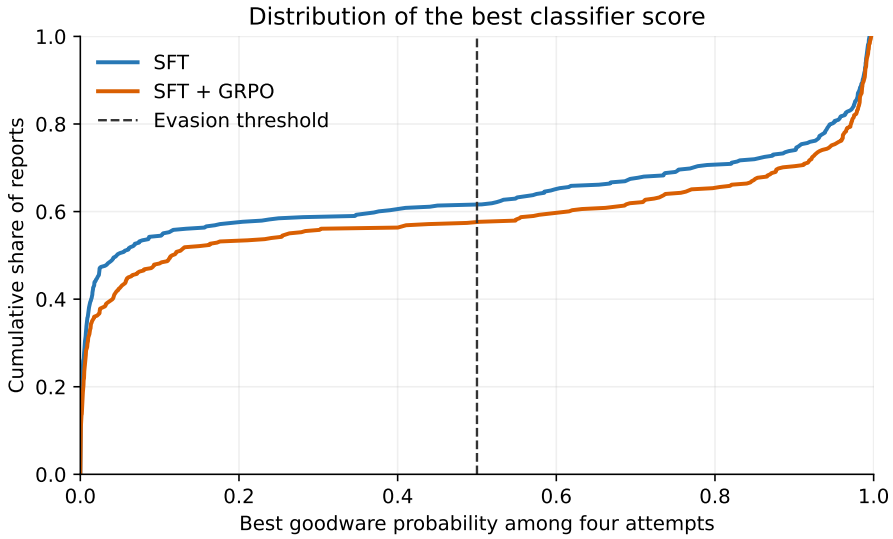



Figure 5.3: Best goodware-score distributions.

Table 5.4: Mean additions by category.

| Category | SFT | SFT + GRPO |
|---|---|---|
| `read_files` | 2.94 | 3.07 |
| `read_keys` | 2.15 | 2.19 |
| `keys` | 2.10 | 1.91 |
| `executed_commands` | 1.27 | 1.63 |
| `mutexes` | 0.66 | 0.75 |

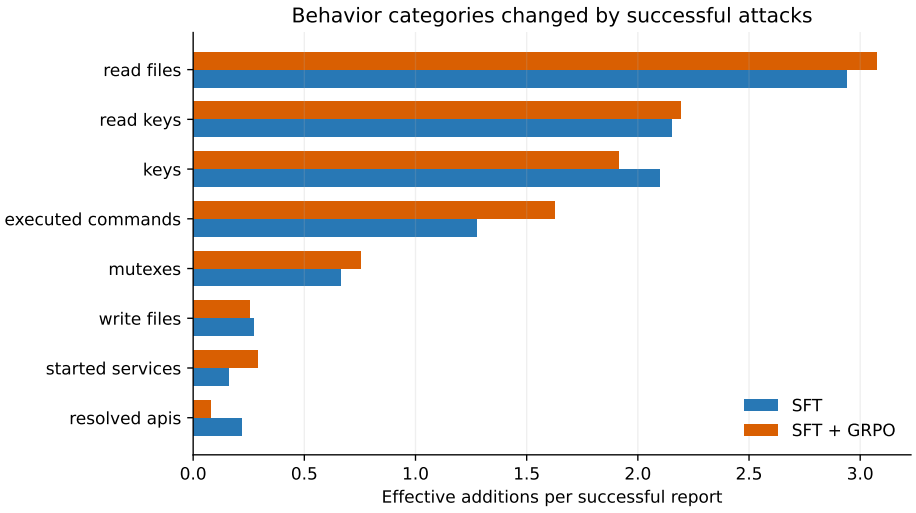



Figure 5.4: Effective additions by category.

number of effective additions, but GRPO selected combinations that moved the MLP score further.

## 5.8 Limitations

The first limitation is the capability of the language models used to generate and learn the attacks. Due to our significant resource constraints, the initial adversarial dataset was produced only by a smaller locally available teacher model. Consequently, the SFT model was trained on a relatively small set of successful strategies and could primarily learn attack patterns represented in those examples.

Model choice is important in cybersecurity tasks, with benchmarks such as CyBench showing stronger frontier models solving tasks that remain beyond weaker models [46]. Access to a stronger teacher could therefore have produced a larger and more diverse set of successful adversarial examples, although this was not possible here.

The second limitation is the single-turn evaluation itself. Each generated candidate is evaluated only after generation, so the language model never sees the effect of its proposed changes. A modification that moves the classifier substantially toward goodware but does not cross the decision threshold is therefore treated as useless. This limitation motivates the multi-turn evaluator introduced in Chapter 6.

# Chapter 6

# Scaling the Adversarial Evaluator

## 6.1 Introduction

Chapter 5 showed that a language model can learn to produce report modifications that evade the MLP. Supervised fine-tuning reached an evasion rate of 38.62%, and classifier-guided GRPO increased it to 42.59%. However, the attacker never saw what happened after its own proposal. In the current single-turn system, any useful movement is discarded, even though it could've been on the brim of shifting the label. An iterative system can keep the change, show the new score to the model, and ask it to continue. Work on language-model agents has shown that interleaving reasoning, action, and feedback can improve tasks that cannot be solved reliably in one generation [42, 30].

## 6.2 Multi-Turn Feedback Loop

The new attacker begins by scoring the unmodified report. Reports already classified as goodware are skipped because there is no malware decision left to evade. For an accepted report, the initial state is

$$x_0 = x, \qquad s_0 = p_{\text{goodware}}(x_0). \tag{6.1}$$

At turn $t$, the prompt contains the original report, the initial score, the current score $s_t$, and the cumulative diff $\Delta_t$. The language model generates four candidate diffs. Each candidate is applied to the current report and scored by the MLP. The evaluator keeps the candidate with the largest goodware probability:

$$\delta_t^* = \arg\max_{\delta \in C_t} p_{\text{goodware}}\left(A(x_t, \delta)\right), \tag{6.2}$$

where $C_t$ is the set of candidates produced at turn $t$. If the best candidate improves the score, it is accepted and merged into the cumulative diff. The modified report becomes the starting point of the next turn.

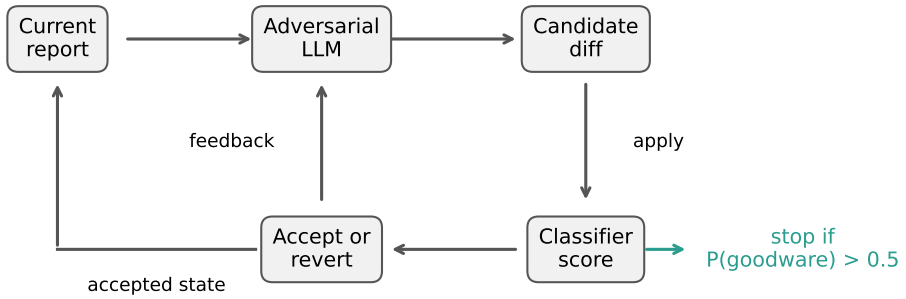



Figure 6.1: Multi-turn feedback loop.

If every candidate reduces the goodware probability, the system reverts the change and generates another attack. If the retry also reduces the score, the attack stops for that report. This follows the same general idea used by language-model agents that revise an action after observing its result [42, 30].

## 6.3 Training

The feedback system used a new adversarial dataset generated through iterative attacks. A successful example could now contain the original report, the teacher's reasoning, and the cumulative diff collected over more than one step. After token and format checks, the cached supervised dataset contained 335 examples.

The new SFT model used the same main settings as the previous experiments: Qwen3-8B in 4-bit form, LoRA rank 16, a 32,000-token sequence limit, and 600 update steps. The model was then trained again with GRPO, which again used the MLP goodware probability as its main dense reward.

## 6.4 Evaluation

The evaluation selected 200 malware reports. Before generation, every report was embedded and scored by the MLP. Each turn sampled four completions at temperature 0.9. Prompts were limited to 32,000 tokens and each completion to 16,000 tokens. Up to five turns and one retry per harmful turn were allowed. vLLM was used to serve the long-context model efficiently [18].

An evasion was successful when a modified report reached $p_{\text{goodware}} > 0.5$. The evaluator saved the initial score, final score, complete score progression, cumulative diff, number of turns, and all generated completions. This makes it possible to study the attack as a trajectory instead of only reporting the final label.

TABLE 6.1: Multi-turn evasion results.

| Outcome | Reports | Share |
|---|---|---|
| Successful evasion | 165 | 86.39% |
| Unsuccessful attack | 26 | 13.61% |
| Total | 191 | 100.00% |

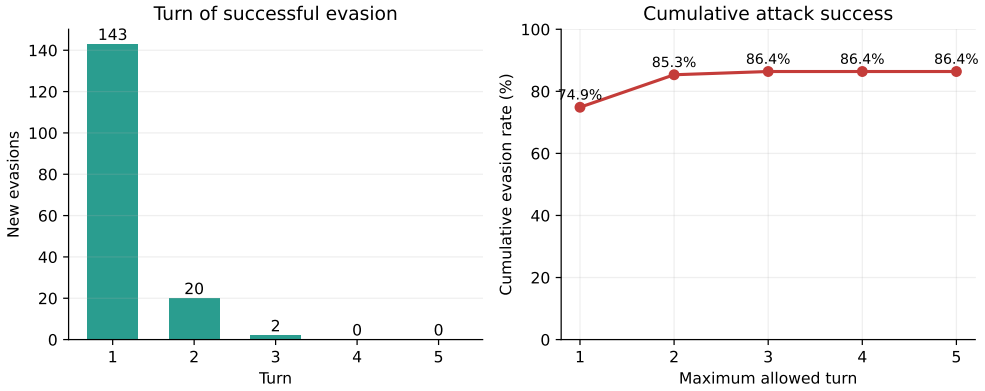



FIGURE 6.2: Evasions by turn.

The multi-turn evaluator evaded 165 of the 191 reports. This gives an evasion rate of 86.39%. Twenty-six reports remained classified as malware.

As shown in Figure 6.2, most reports were evaded immediately: 143 of the 191 attacks succeeded during the first turn. The second turn was still useful because classifier feedback helped another 20 reports cross the decision boundary, increasing the cumulative evasion rate from 74.87% to 85.34%. After this point, the return was very small. Only two additional reports were evaded in the third turn, and turns four and five produced no new successes. The figure therefore shows that feedback mattered mainly as one opportunity to correct the first proposal, rather than as a long sequence of revisions.

The high first-turn result shows that the retrained model was already much stronger than the earlier single-turn model. At the same time, several reports were rescued by classifier feedback after the first proposal had failed. The second turn produced nearly all of this additional benefit. A five-turn budget was useful as a safety limit, but it was not necessary for most reports.

The score trajectories from Figure 6.3 show the difference between successful and failed attacks more clearly. Successful reports started with a small mean goodware probability and rose exponentially. The failed reports started much further from the boundary and struggle to rise.

Even the failed group usually moved toward goodware. However, the failed samples began with extremely small goodware probabilities, and five turns were not enough to cross the boundary. This suggests our baseline classifier already learned to identify several key behaviors that the defined attack model cannot evade.

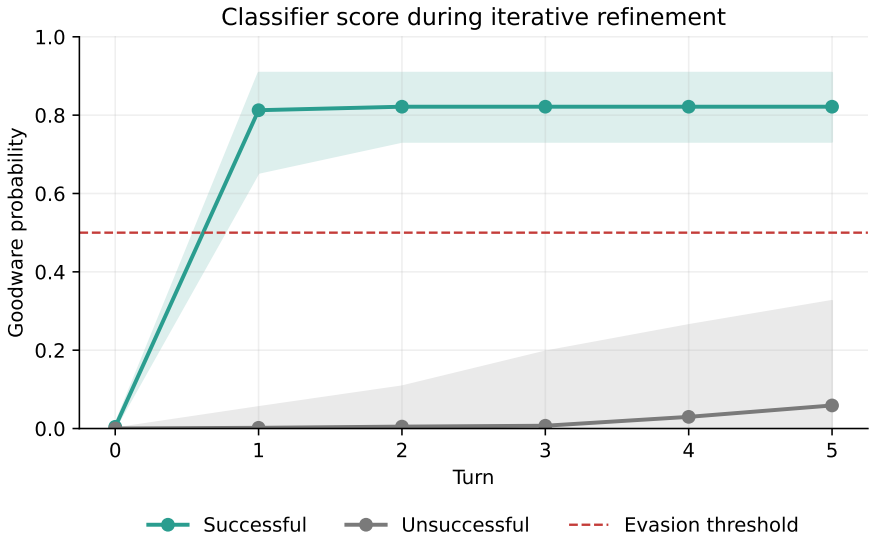



Figure 6.3: Classifier-score trajectories.

Table 6.2: Final perturbation budgets.

| Attack outcome | Mean | Median | Maximum |
|---|---|---|---|
| Successful | 60.1 | 47 | 394 |
| Unsuccessful | 145.1 | 33 | 494 |

Table 6.2 summarizes the final perturbation budgets, while Figure 6.4 shows their distributions. Successful attacks used a mean of 60.1 operations and a median of 47. Failed attacks had a smaller median of 33 but a much larger mean of 145.1, as all five turns were used.

These results show that a larger perturbation is not automatically a better attack. Some failed reports contained more than twice as many operations on average as successful reports. What was changed, and which values were used, therefore mattered more than the raw size of the diff.

## 6.5 Robustified Classifier

The successful adversarial reports were then used to harden the original MLP. Rather than training a new classifier from scratch, I continued training the baseline for one additional epoch, mixing the original training data with 422 malware reports generated by the adversarial evaluator. These reports preserve the malware label while presenting the classifier with difficult variants of samples it had previously learned to classify [22, 37].

The embedding model and MLP architecture were left unchanged. Fine-tuning used a learning rate of $10^{-4}$ and a batch size of 32. The relatively small adversarial

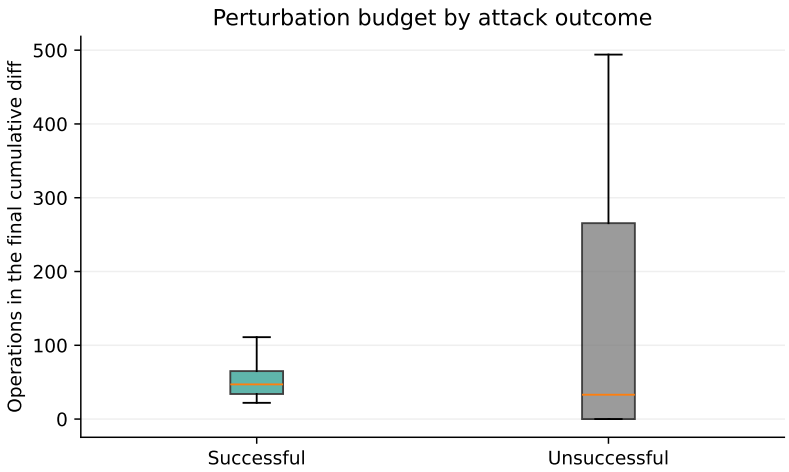



FIGURE 6.4: Perturbation-budget distributions.

TABLE 6.3: Clean classification before and after adversarial hardening.

| Model | Accuracy | Balanced acc. | Macro-F1 | Malware prec. | Malware recall | ROC-AUC |
|---|---|---|---|---|---|---|
| Baseline | 0.9877 | 0.9864 | 0.9864 | 0.9906 | 0.9906 | 0.9987 |
| Robustified | 0.9985 | 0.9972 | 0.9981 | 0.9980 | 1.0000 | 1.0000 |

set was therefore used as an additional hardening signal rather than as a replacement for the original training distribution.

Table 6.3 and Figure 6.5 show that the additional adversarial training did not trade away clean performance. Instead, the robustified model slightly improved across all reported metrics.

The improvement is also visible at the family level. As shown in Figure 6.6, the largest change occurs for Virlock, while performance on Vobfus and Shipup remains effectively perfect. Goodware classification also improves slightly. The hardening stage therefore improved performance on the clean test distribution in addition to exposing the model to the adversarial variants generated in the preceding experiment.

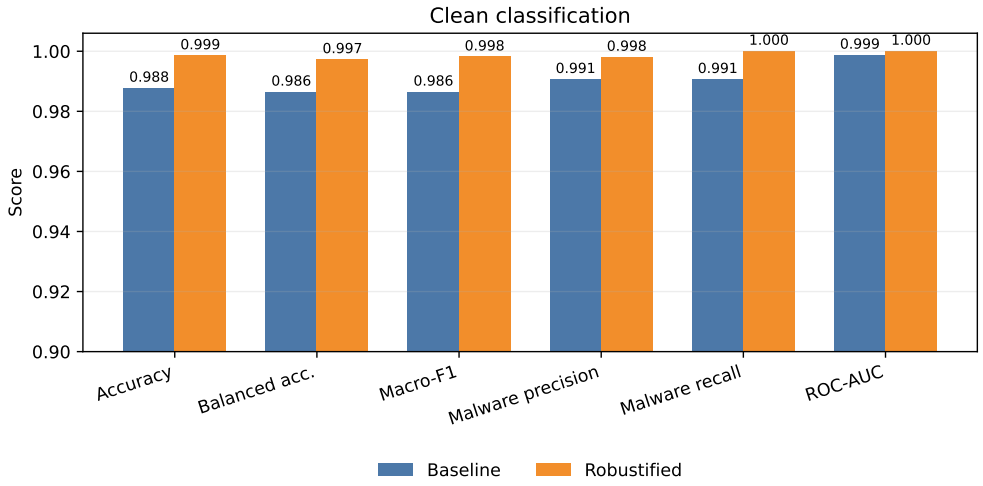



Figure 6.5: Clean classification performance before and after adversarial hardening.

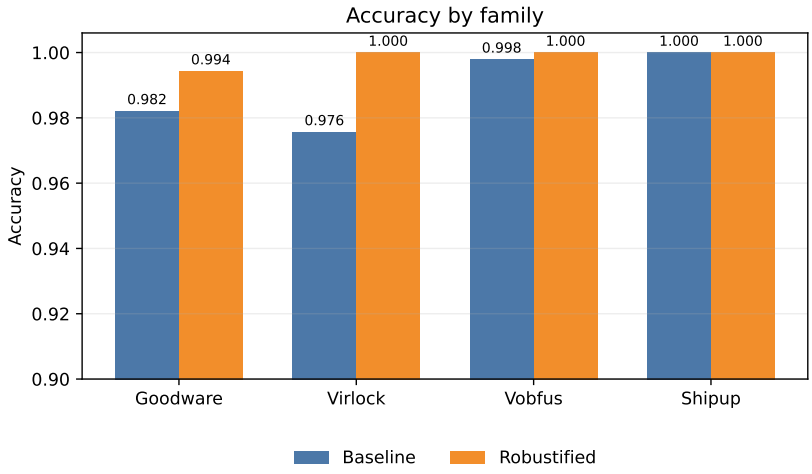



Figure 6.6: Classification accuracy by class and malware family before and after adversarial hardening.

# Chapter 7

# Generating Explanations

## 7.1 Introduction

The previous chapters produced two versions of the malware classifier: the original MLP and the classifier obtained after adversarial hardening. The remaining problem is how to present the classifier decision to a human analyst. The attribution methods in Chapter 4 identify influential categories or behavioral entries, but they do not combine this evidence into a readable explanation of why the observed behavior supports the prediction. I therefore investigate whether a language model can perform this second task.

A first approach trains the language model to both classify the report and explain its own decision. As shown below, this produces readable output but does not reproduce the accuracy of the dedicated MLP. The final design therefore separates the responsibilities: the MLP determines the prediction, while the language model generates an explanation conditioned on the report and classifier representation.

## 7.2 Teacher Explanation Data

Knowledge distillation trains a smaller student model using targets produced by a larger teacher model [13]. I used this approach to create the explanation dataset: a larger model first analysed each behavior report, and its response was later used as the supervised target for Qwen3-8B.

The selected teacher was GLM-5.2. Its official model card reports performance close to proprietary frontier models on several general reasoning benchmarks - for example, it reports 91.2 on GPQA-Diamond, compared with 93.6 for both Claude Opus 4.8 and GPT-5.5 [45]. These are general benchmark results reported by the model developer, not a malware-analysis evaluation, but they supported using GLM-5.2 as a strong reasoning teacher within budget constraints.

To make the analysis easier to build and read, the teacher first divided its observations into three groups: suspicious behavior, benign behavior, and missing or weak evidence. It then combined these observations into one final explanation. The separate lists make the result easier for an analyst to scan, while the final para-

graph gives one coherent justification for the decision. This decomposition was also intended to help the model consider evidence both for and against its label before writing the final explanation. Reports were sampled by class and length so that the teacher did not receive only short or majority-class examples.

The teacher produced one JSON object with the following schema:

```
{
  "label": "malware | benign",
  "confidence": 0.0,
  "suspicious_behaviors": [],
  "benign_behaviors": [],
  "missing_or_weak_evidence": [],
  "explanation": ""
}
```

Due to resource constraints, teacher explanations were generated for the training and held-out reports. GLM-5.2 received the complete behavior reports together with clear instructions to identify the required behaviors and summarize them in an explanation. The complete prompt is provided in subsection A.1.2.

The teacher reached an accuracy of 87% in the initial evaluation. Incorrect teacher decisions were not useful targets for a classifier that already performed better. Later fusion datasets therefore kept the explanation text only when the teacher label agreed with the MLP label.

## 7.3 Initial Supervised Fine-Tuning

The first student was Qwen3-8B, loaded in 4-bit form and fine-tuned with LoRA. The full teacher response was used as the target.

At validation, the SFT reached 47.42% label accuracy, and produced directly parseable JSON for 59.03% of them. These values showed that explanation distillation alone did not preserve the accuracy of the MLP. The central problem was architectural: the student was asked to learn the classifier again from language-model loss. Given this, I concluded the need to add the robust MLP to the explainer LLM's architecture.

We conclude then that this experiment did not turn the language model into a reliable malware classifier. The model was effectively being asked to relearn from language-model supervision a decision boundary that the dedicated MLP had already learned much more accurately. I therefore retained the explanation behavior learned during SFT, but removed classification as a responsibility of the language model in the final architecture.

The initial SFT experiment exposed an unnecessary difficulty in the original formulation. The language model was asked to infer the malware label from the report and then explain that label, even though a dedicated classifier had already learned the classification task much more accurately. The final system therefore separates these responsibilities: the MLP determines the class, while the language model is conditioned on that decision and generates its explanation.

I use two complementary forms of conditioning. First, generation is explicitly initialized with the classifier prediction. Before the model generates its own reasoning, the following text is inserted into the reasoning block:

```
<think>
Classifier signal: classifier predicts {label}
with confidence {confidence}.
Treat this classifier decision as authoritative.
```

The label and confidence are taken directly from the frozen MLP. This changes the task presented to the language model: rather than first deciding whether the report is malicious, it begins from the classifier decision and must explain that decision using the behavior described in the report.

The explicit signal alone, however, contains very little information about *why* the classifier produced its prediction. Two reports can receive the same label and similar confidence while reaching that result from very different behavioral evidence. I therefore additionally expose internal representations from the classifier to the language model. The explicit conditioning provides the high-level decision that should be explained, while the learned fusion path gives the language model access to richer information produced while computing that decision.

## 7.4 Fusion Architecture

Figure 7.1 shows the final, complete flow. The frozen embedding model and frozen MLP form the upper branch. The language-model prompt forms the lower branch. They meet through a compact classifier memory that is read by cross-attention in two decoder layers.

### 7.4.1 Log memory

The embedding model returns one 1,024-dimensional hidden state for every input token. Passing the complete sequence to each fusion layer would make the cross-attention cost grow with the length of the report. I therefore use 16 learned query vectors that attend over the embedding sequence and produce 16 fixed-size summary tokens.

This design follows the same pattern used by multimodal language models. Flamingo uses a Perceiver Resampler with learned latent queries to convert a variable number of visual features into a fixed number of tokens before they are read by gated cross-attention layers in a frozen language model [16, 1]. In this thesis, the input is not an image, but the purpose is nevertheless the same: create a small, fixed memory that the language model can access without attending directly to every encoder token at every fusion layer.

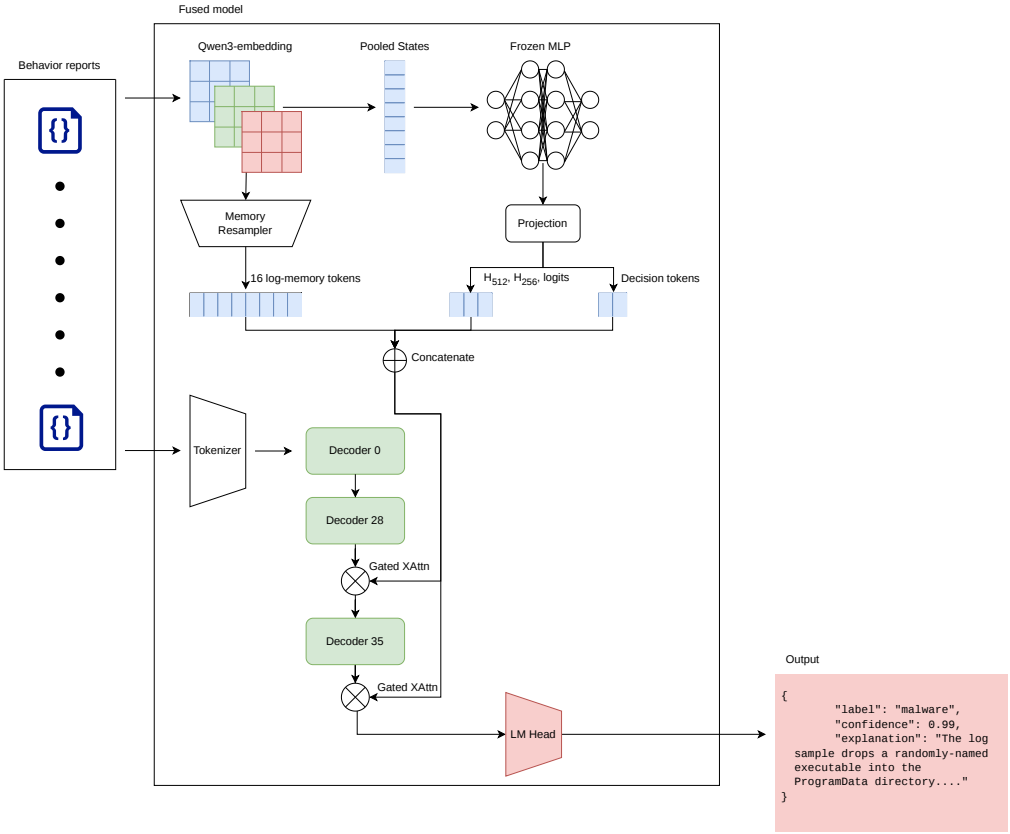



Figure 7.1: Classifier–LLM fusion architecture.

### 7.4.2 MLP and decision memory

The pooled embedding is normalized and passed through the same MLP used for classification. Three internal results are retained: the 512-dimensional hidden state, the 256-dimensional hidden state, and the two output logits. Each is projected into one language-model memory token.

Two additional tokens encode the final decision. Their input contains the benign probability, malware probability, confidence assigned to the predicted class, and the absolute probability margin. The complete classifier memory therefore contains

$$N_{\text{memory}} = 16 + 2 + 1 + 1 + 1 = 21 \tag{7.1}$$

vectors. This representation gives the language model access to both the final answer and intermediate classifier representations used to compute it.

### 7.4.3 Gated cross-attention

Cross-attention is inserted after self-attention in decoder layers 28 and 35. The decoder states form the queries, while the 21 classifier-memory vectors form the keys and values. The attention width is 128 with four heads. Its output is projected back to the 4,096-dimensional language-model width and added through a learned gate:

$$H' = H + \sigma(g)\ \mathrm{CrossAttn}(H, M), \tag{7.2}$$

where $H$ denotes the current decoder states, $M$ is the classifier memory, and $g$ is a scalar learned separately for each fusion layer.

Only the resampler, projections, auxiliary head, cross-attention layers, and gates are trained in the final fusion stage. The embedding model, the MLP, the Qwen3-8B base weights, and the previously trained explanation adapter remain frozen. This protects the classifier boundary and limits the number of updated parameters.

These architectural values were selected as engineering choices under the available memory budget rather than through an exhaustive hyperparameter search. Sixteen resampled tokens provide a fixed and substantially smaller representation than attending to the complete report at every fusion layer, while inserting cross-attention only in two late decoder layers limits the number of trainable parameters.

### 7.4.4 Auxiliary classifier

The language-model loss alone does not require the constructed memory to preserve information about the classifier decision. In principle, the trainable resampler and projection layers could learn representations that are useful for reproducing the teacher explanation while discarding part of the discriminative signal contained in the classifier branch.

This creates the need to add an auxiliary classification objective to the memory path, to encourage intermediate representations to retain information relevant to the predictive task [19]. A closely related approach is used by Gspell+, where the projected representation supplied to an LLM is additionally optimized to preserve information about the prediction of the model being explained [3].

The 21 classifier-memory tokens are mean-pooled into a single representation,

$$\bar{M} = \frac{1}{N_{\mathrm{memory}}} \sum_{i=1}^{N_{\mathrm{memory}}} M_i, \tag{7.3}$$

and a linear auxiliary head maps this representation to two class logits,

$$\hat{y}_{\mathrm{aux}} = W_{\mathrm{aux}}\bar{M} + b_{\mathrm{aux}}. \tag{7.4}$$

## 7.5 Training

The original explanation dataset was used for training, with a caveat: the teacher model and the MLP did not always assign the same label. Training on both signals would ask the fused model to use the classifier memory and contradict it at the same time. As such, examples where the teacher label and classifier label disagree are removed from both splits.

After disagreement filtering, the new dataset contained 12,935 training examples and 677 evaluation examples. The filter removed 897 training examples and 51 evaluation examples whose teacher label disagreed with the frozen classifier.

The main loss is ordinary SFT's next-token cross-entropy over the unmasked assistant tokens, to which I add the previously defined auxiliary classification loss:

$$\mathcal{L} = \mathcal{L}_{\text{SFT}} + 0.2\,\mathcal{L}_{\text{aux}}. \tag{7.5}$$

The final configuration uses one epoch, batch size one, four-step gradient accumulation, a learning rate of $5 \times 10^{-5}$, 100 warm-up steps, BF16 computation, gradient checkpointing, and an 8-bit paged AdamW optimizer. Samples are ordered by length during training to reduce padding and make memory use more predictable.

Figure 7.2 summarizes the optimization of the fusion components. The total training loss decreases rapidly during the beginning of training and then stabilizes. The auxiliary classification loss similarly falls to a low value, while the auxiliary head approaches the predictions of the frozen classifier. The gradient norm settles after an initial transient without showing unstable growth.

The cross-attention gates remain small throughout training. The classifier memory therefore enters the decoder as a limited residual signal rather than overwriting the pretrained language-model representation. These diagnostics indicate that the added fusion components can be optimized stably; whether this conditioning actually improves the generated explanations is evaluated separately in Chapter 8.

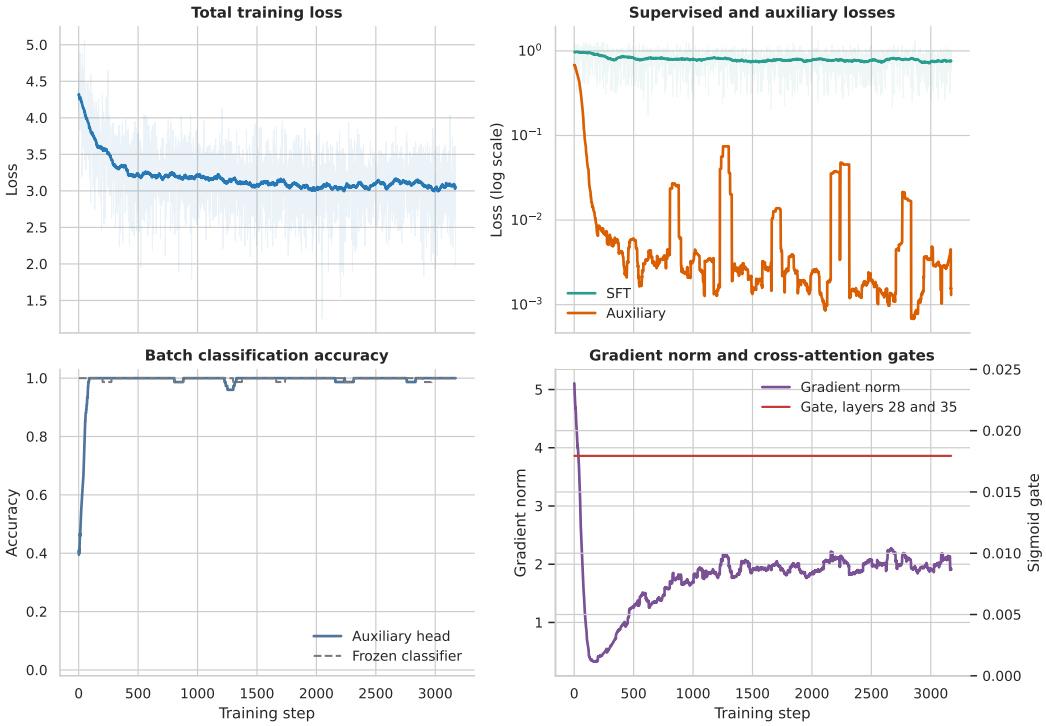



FIGURE 7.2: Fusion-training diagnostics.

# Chapter 8

# Evaluation and Discussion

This chapter evaluates the complete classification and explanation pipeline. I will analyze the models from two viewpoints. First, the quantitative part compares the labels produced at the main stages of the project and defines the faithfulness test used for the explanations. The qualitative part then selects concrete reports for closer manual analysis. This separation is important because a model can produce the correct label while giving a weak explanation, or produce a fluent explanation for an incorrect label [38].

## 8.1 Quantitative Evaluation

### 8.1.1 Experimental setup & metrics

I compared four versions of the system: *Base Qwen3-8B* is the untrained language model prompted to classify a report and return JSON. *SFT-only* is the same language model after explanation distillation, without access to an MLP. *Baseline fusion* combines the language model with the original MLP, while *robust fusion* uses the MLP obtained after the adversarial robustification evaluated in Section 6.5.

I used two groups of metrics for the quantitative evaluation. First, I measured the classification performance of each pipeline using accuracy, precision, recall, and F1-score. These metrics allow us to compare whether adding explanation supervision affects the predictive performance of the classifier.

Second, I evaluated the faithfulness of the generated explanations. An explanation is faithful when the evidence it identifies as relevant to a prediction corresponds to information that actually influences that prediction. I evaluated this using a perturbation-based comprehensiveness test [2, 31]. For each generated explanation, an independent LLM removes from the original sandbox report the entries identified by the explanation as supporting the prediction. The modified report is then classified again by the same pipeline. If the removed evidence was important to the original decision, its removal should decrease the confidence in the predicted class.

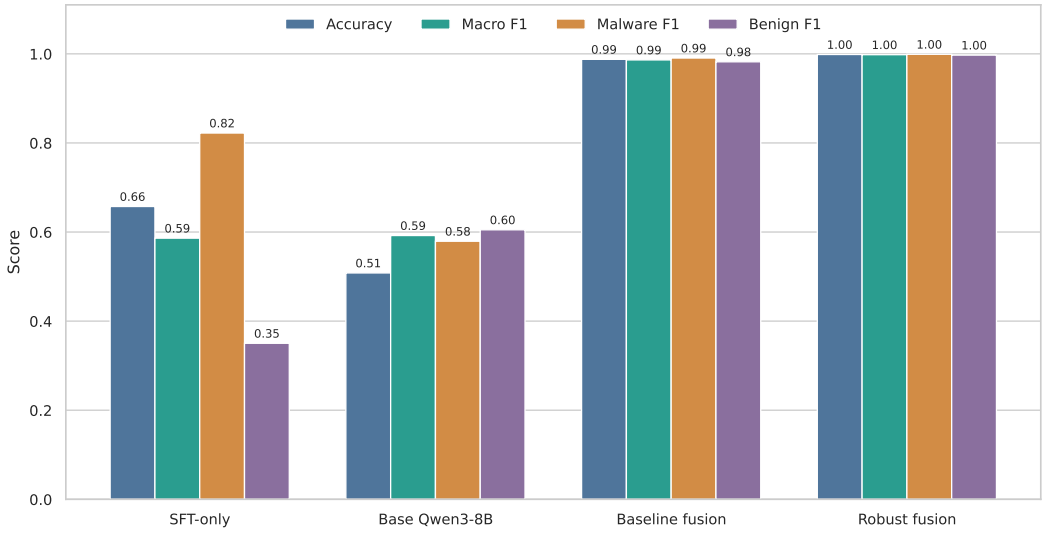



Figure 8.1: Classification scores.

### 8.1.2 Classification results

Figure 8.1 shows a large difference between a language model that makes the decision itself and a language model connected to the classifier.

Predictably, the base, untrained model is the weakest, performing little better than chance, while explanation distillation substantially improves the performance of the `SFT-only` model. However, both of these variants remain considerably less reliable than the fused systems.

The largest weakness of the `SFT-only` model is its performance on benign samples. Although it performs better than the base model, it largely does so by developing a strong preference for the malware label, achieving a good performance on malware samples while struggling to recognize benign software.

In contrast, both fused variants closely reproduce the decisions of their underlying MLP classifiers, which is expected because the classifier prediction determines the final class decision. The purpose of the language model in these variants is therefore to generate an explanation for a classifier decision, rather than to independently recover the class label.

We can conclude therefore that, under the chosen training setup, relying on the language model alone is insufficient for reliable classification. Adding the dedicated classifier preserves the performance while allowing the language models to generate explanations.

### 8.1.3 LLM judge

The classification results do not measure whether an explanation is clear or useful. As such, I performed an automatic evaluation of the explanations with `qwen3.6:27b`

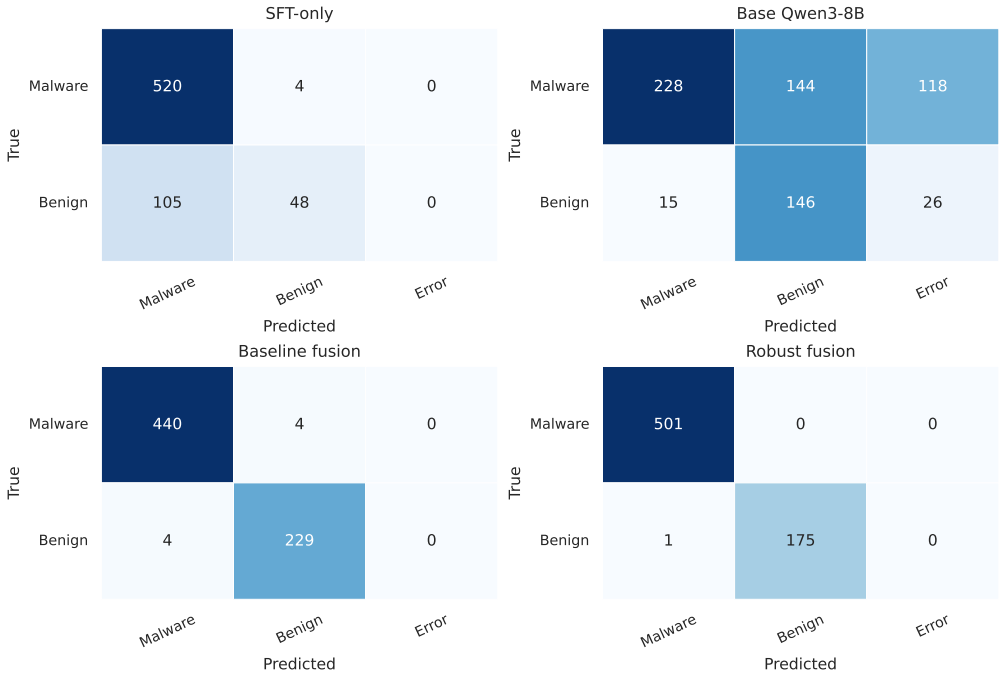



FIGURE 8.2: Prediction counts.

TABLE 8.1: Judge scores.

| Model | Understandability | Trust | Insight | Satisfaction | Usability | Mean |
|---|---|---|---|---|---|---|
| Base Qwen3-8B | $3.71 \pm 0.63$ | $2.74 \pm 1.16$ | $1.96 \pm 0.69$ | $2.17 \pm 0.88$ | $2.14 \pm 0.83$ | $2.54 \pm 0.72$ |
| SFT-only | $4.26 \pm 0.69$ | $3.16 \pm 1.38$ | $2.88 \pm 1.05$ | $3.08 \pm 1.31$ | $3.11 \pm 1.30$ | $3.30 \pm 1.06$ |
| Baseline fusion | $4.21 \pm 0.67$ | $3.39 \pm 1.27$ | $3.17 \pm 0.96$ | $3.28 \pm 1.08$ | $3.33 \pm 1.08$ | $3.47 \pm 0.94$ |
| Robust fusion | $\mathbf{4.45 \pm 0.61}$ | $\mathbf{3.58 \pm 1.20}$ | $\mathbf{3.45 \pm 0.89}$ | $\mathbf{3.66 \pm 1.03}$ | $\mathbf{3.69 \pm 1.02}$ | $\mathbf{3.77 \pm 0.89}$ |

as an LLM judge. I selected and adapted five human-centered evaluation dimensions from the framework used by Baghershahi et al. [4].

Understandability measures whether the text clearly explains the decision. Trustworthiness measures whether its claims are supported by the behavior report. Insightfulness measures whether the explanation connects concrete operations to their security meaning instead of only repeating the input. Satisfaction measures whether the strongest available evidence is covered in a complete and meaningful way. Finally, usability measures whether the text would help an analyst inspect the decision.

Every dimension was scored from 1 to 5. A score of 3 represents an adequate explanation with important limitations.

For each report, the judge received the original behavior summary, the label predicted by each system, and the corresponding explanation. Table 8.1 reports the mean and standard deviation across reports.

The explanation quality shows constant improvement as additional supervision was introduced into the pipeline. The base language model obtains the lowest scores, with its weakest dimension being insightfulness (1.96). This indicates that the gen-

Table 8.2: Explanation faithfulness under deletion-based perturbation.

| Model | Faithfulness rate (%) | Mean edits |
|---|---|---|
| Base Qwen3-8B | 69.39 | 168.41 |
| SFT-only | 66.10 | 70.51 |
| Baseline fusion | 71.19 | 58.66 |
| Robust fusion | 81.36 | 103.00 |

erated text often does not connect observed behavior to its security significance. Explanation distillation brings the most substantial improvement across all five dimensions, raising the mean score to 3.30.

Both fusion variants outperform the language-only systems, with robust fusion achieving the highest scores on every evaluated dimension. The largest gains over baseline fusion occur in satisfaction and usability, suggesting that representations from the robust classifier allow the language model to produce explanations that cover more relevant evidence and are more useful for interpreting the decision. These results indicate that the classifier representation also affects the quality of the generated explanation.

### 8.1.4 Explanation faithfulness

Classification correctness does not establish that the generated explanation uses the evidence that actually supports the decision. I therefore use a single-turn deletion test. A separate model maps the generated explanation back to exact entries in the behavior report. These entries are removed, the modified report is passed through the same evaluated system, and the final label is measured again. This follows the general idea of deletion-based explanation evaluation: removing evidence identified as important should have a measurable effect on the prediction [43, 39]. I additionally extracted the mean number of report entries removed by the deletion model.

Table 8.2 shows that the base model reaches a 69.39% faithfulness rate, but requires by far the largest intervention, with 168.41 removed entries on average. This suggests that the evidence cited by the untrained model is broad and not well aligned with the evidence driving its prediction.

Explanation distillation reduces the average deletion set to 70.51 entries, although its faithfulness rate is slightly lower at 66.10%. Both fused models exceed the SFT-only rate. Baseline fusion reaches 71.19% with the smallest mean deletion set, while robust fusion reaches the highest rate, 81.36%, with 103.00 edits on average.

These results suggest that classifier fusion creates a stronger link between the generated explanation and the evidence used by the complete system. This effect is strongest in the robust model, although its larger deletion set than baseline fusion also indicates that its explanations may cover more evidence.

## 8.2 Qualitative Evaluation

Automatic scores cannot show whether an explanation uses precise technical language, separates observed facts from interpretation, or introduces behavior that is absent from the report. This is important for generated explanations, as fluent text is not necessarily faithful to the model decision [38]. I therefore manually compare three reports selected to illustrate different behaviors of the evaluated systems.

### Case 1: Benign registry activity

The first report contains only a small number of registry reads and one resolved C runtime API. Base Qwen labels it as malware, while both fused models classify it as benign.

Listing 8.1: Relevant behavior entries for report `e3c212d5...`

```
registry_reads:
  - DisableUserModeCallbackFilter
  - HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\GRE_Initialize\DisableMetaFiles

resolved_apis:
  - msvcrt.dll.___lc_codepage_func

file_operations: none
executed_commands: none
services: none
```

Listing 8.2: Model outputs for report `e3c212d5...`

```
Base Qwen3-8B:
  label: malware
  confidence: 0.90
  explanation: The sample exhibits suspicious registry key read and write behaviors
     targeting specific Windows registry locations associated with system configuration.
     The absence of executed commands or file operations weakens the benign context, but
     the targeted registry modifications are commonly seen in malware for persistence or
     system modification.

Baseline fusion:
  label: benign
  confidence: 0.96
  explanation: The log contains minimal activity consisting of reading standard Windows
     registry keys under 'GRE_Initialize' and querying system configuration via '
     DisableUserModeCallbackFilter'. These reads are typical for applications initializing
     their graphical environment using standard C runtime libraries like msvcrt.dll. There
     are no file operations, process executions, network activities, service creations, or
     other malicious indicators.

Robust fusion:
  label: benign
  confidence: 0.98
  explanation: The log contains only minimal, non-malicious behaviors. The sample reads
     standard Windows registry keys associated with the Graphics Device Interface (GDI) and
      system configuration, specifically 'GRE_Initialize' and '
     DisableUserModeCallbackFilter'. These are typical queries made by legitimate
     applications during initialization. There are no file operations, process executions,
     network communications, or persistence mechanisms present. Based solely on the
     explicitly observed behaviors, the sample exhibits no malicious indicators.
```

The base model incorrectly turns registry reads into both "read and write behaviors" and later into "registry modifications". It then associates these non-existent modifications with persistence. Both fused models instead preserve the distinction between registry access and modification and correctly note the absence of stronger malicious indicators.

### Case 2: Correct label, unsupported explanation

The second report is a ShipUp malware sample. Both fused models classify it as malware, allowing the comparison to focus on explanation quality rather than classification accuracy.

Listing 8.3: Relevant behavior entries for report `d612d8f0...`

```
file_reads:
  - C:\Windows\System32\55222
  - C:\Users\John\AppData\Local\Temp\d612d8f059ee10512d54c156.exe
  - \Device\KsecDD

file_writes:
  - C:\ProgramData\Mozilla\ysokqrk.exe

registry_reads: 29 entries
registry_writes: none
executed_commands: none

resolved_apis:
  - VirtualAlloc
  - MoveFileExW
  - IsDebuggerPresent
  - UnhandledExceptionFilter
  - RegSetValueExW
```

Listing 8.4: Fused-model outputs for report `d612d8f0...`

```
Baseline fusion:
  label: malware
  confidence: 1.00
  explanation: The sample exhibits several strong indicators of malicious behavior. It
     creates a randomly-named executable in ProgramData/Mozilla, which is highly unusual
     and often used by malware for persistence. The execution of ysokqrk.exe from
     ProgramData suggests it may be a dropper or downloader. Additionally, the sample
     writes to MachineGuid under Cryptography, modifies language-pack-related HKLM keys,
     and resolves numerous API calls typical of malware initialization.

Robust fusion:
  label: malware
  confidence: 1.00
  explanation: The log exhibits multiple strong indicators of malicious behavior. The
     sample drops a randomly-named executable into the ProgramData directory (C:\
     ProgramData\Mozilla\ysokqrk.exe), a common persistence mechanism for malware. It reads
      the kernel security device \Device\KsecDD, which is typically associated with
     cryptographic operations. The resolved APIs suggest dynamic library loading, memory
     allocation, and file I/O, consistent with injection or payload delivery. Anti-sandbox
     checks (IsDebuggerPresent, UnhandledExceptionFilter) and file system manipulation (
     GetTempPathW, MoveFileExW) are also present. While some registry reads are benign (
     locale, language packs), the combination of dropped executables, cryptic device access
     , and anti-analysis indicators strongly suggests malware.
```

The baseline-fusion explanation contains several unsupported claims. It states that `ysokqrk.exe` is executed despite the absence of executed commands, and describes registry modifications despite the report containing no registry writes. Resolving `RegSetValueExW` only indicates the availability of such functionality and does not prove that it was used.

The robust-fusion explanation is better grounded, correctly identifying the dropped executable, `\Device\KsecDD` access, and relevant APIs. It still occasionally treats capability as observed behavior: resolving `IsDebuggerPresent`, for example, does not establish that an anti-debugging check was actually performed. Robust fusion therefore reduces, but does not eliminate unsupported interpretations.

### Case 3: Grounded malware explanation

The final report is a short malware example. It writes and executes `C:\2828222.exe` and resolves several APIs associated with process manipulation.

Listing 8.5: Relevant behavior entries for report `0560a1c0...`

```
file_reads:
  - C:\Users\John\AppData\Local\Temp\09c4b74aaa7e2773c5e652c2.exe

file_writes:
  - C:\2828222.exe

executed_commands:
  - c:\2828222.exe

resolved_apis:
  - OpenProcess
  - VirtualAllocEx
  - WriteProcessMemory
  - CreateProcessA

registry_writes: none
services: none
```

Listing 8.6: Robust-fusion output for report `0560a1c0...`

```
Robust fusion:
  label: malware
  confidence: 1.00
  explanation: The log shows the sample dropping and executing an executable (c:\2828222)
     from a randomly-named Temp file (09c4...). This is a classic dropper/loader pattern
     commonly associated with malware. The resolved APIs include process manipulation
     functions (OpenProcess, VirtualAllocEx, WriteProcessMemory), indicating potential
     injection or execution of malicious code. While no direct network communication or
     persistence mechanisms are observed, the combination of dropping and executing an
     executable from a Temp file is highly suspicious and consistent with malware behavior.
```

Here, the robust-fusion explanation closely follows the observed behavior. It correctly identifies the written and executed executable and treats the resolved process-manipulation APIs as evidence of *potential* injection rather than claiming that injection was directly observed. It also correctly notes the absence of persistence and network behavior.

Overall, the examples follow the same trend as the automatic evaluation. The base model can produce fluent explanations containing behavior absent from the report, while the fused models are generally more specific and grounded. Robust fusion performs best in these examples, although it can still over-interpret capabilities implied by resolved APIs.

## 8.3 Discussion

The results show a clear benefit to separating classification from explanation generation. The language-only models are not sufficiently reliable classifiers, while the fused models preserve the performance of the underlying MLP. At the same time, the language model can use the classifier output and representation to produce a natural-language explanation of the decision.

This result is important for malware analysis as a classifier that outputs only a score provides limited information about why a sample was considered malicious. Cybersecurity analysts require more than a classification result; explanations should provide higher-level, easily interpretable information that can be incorporated into existing analysis workflows [33, 23]. An explanation can expose what behaviors were associated by the underlying model with the prediction, allowing an analyst to inspect and trust the decision of the model if it is supported by meaningful evidence.

The explanation evaluation suggested that this additional output is not purely cosmetic - classifier fusion improves both automatic judge scores and faithfulness. The qualitative examples show the same trend: the fused systems are generally more specific and better grounded in observed behavior than the base language model.

The generated explanations should not be treated as a perfect reconstruction of the classifier's reasoning. The qualitative examples show that even the fused models can sometimes over-interpret the available evidence, for example by turning a resolved API into behavior that was not directly observed. Their main value is therefore to make the classifier's decision easier for an analyst to inspect, while the original behavior report remains the final source of evidence.

# Chapter 9

# Conclusions

The central motivation of this thesis is that a malware classifier should provide more than a score. A prediction such as "99% malware" may be useful for automated filtering, but it gives an analyst little information with which to judge whether that confidence is justified. An explanation that connects the decision to concrete observed behavior makes the result more useful for investigation: it allows the analyst to inspect why the model is confident, decide whether the identified evidence is meaningful, and use that information in subsequent analysis rather than treating the classifier as a black box.

The experiments in this thesis show both why this is necessary and that it is feasible. Explainability methods revealed that an accurate classifier can still rely on incidental artifacts, while adversarial evaluation provided another way to expose and reduce such weaknesses. Finally, separating prediction from explanation proved more effective than asking a language model to perform both tasks. A dedicated classifier can retain responsibility for the decision, while a language model conditioned on that classifier can turn the available evidence into a substantially more interpretable explanation. The resulting text is not a replacement for the sandbox report or a perfect reconstruction of the classifier's reasoning, but it provides an analyst with considerably more context than a class probability alone.

The main limitation of this work is the scope of the available data. The dataset contains only a small number of malware families and represents only a limited view of their runtime behavior. In particular, the original AutoRobust samples were executed without network connectivity for safety reasons, which may prevent network-dependent malware from exposing important parts of its behavior [37]. A larger and more diverse dataset, collected from newer samples and richer sandbox environments, would provide a substantially stronger basis for both classifier training and explanation evaluation.

Despite this limitation, the experiments show that useful natural-language explanations can already be generated from the available behavioral evidence. With broader data, the same approach could cover a wider range of malicious behavior and provide more complete explanations. Future work should therefore focus primarily on richer behavioral datasets, stronger grounding of generated claims in exact

sandbox observations, and evaluation with malware analysts to determine whether these explanations improve real investigation workflows.

# Appendices

# Appendix A

# Experimental Material

## A.1 Prompts

### A.1.1 Multi-turn adversarial GRPO

Listing A.1: Abbreviated prompt used for multi-turn GRPO.

```
[SYSTEM]
You are a cybersecurity expert specializing in malware evasion analysis.
Your task is to generate an adversarial perturbation (a "diff" in JSON
format) that, when applied to a malware behavioral log, makes a classifier
think it is "goodware".

You have TWO types of operations available:
1. add: Insert new entries characteristic of legitimate software behavior.
2. edit: Replace existing suspicious/malicious entries with benign-looking
   alternatives that a goodware program would produce.
You are not allowed to edit API calls (resolved_apis), but you CAN add them.

Rules:
1. Output only the JSON diff and nothing else.
2. Use this exact format:
{
  "add": {"category": ["new_entry_1", "new_entry_2"]},
  "edit": {"category": [{"old": "exact_existing_entry",
                         "new": "replacement_entry"}]}
}
3. Valid categories: <VALID_CATEGORIES>
4. Each old value MUST exactly match an entry in the current log state.
5. Replacement and added entries must be realistic goodware behavior.
6. Do not duplicate existing entries.
7. Do not remove entries; only add or replace.
8. Make between 10 and 100 additions and edits to each log.
9. Focus on the most impactful categories.

Category importance:
<IMPORTANCE_RANKING>

Common goodware behavioral entries:
```

```
<GOODWARE_REFERENCE_DATA>

[USER, FIRST TURN]
## Initial Input
<REPORT_JSON>

## Initial Classifier Score: P(goodware) = <INITIAL_SCORE>
## Working Diff (accumulated changes so far): {}
## Current Classifier Score: n/a

Generate a JSON diff to push P(goodware) above 0.50.

[USER, LATER TURN]
## Initial Input
<REPORT_JSON>

## Initial Classifier Score: P(goodware) = <INITIAL_SCORE>
## Working Diff (accumulated changes so far): <ACCUMULATED_DIFF>
## Current Classifier Score: P(goodware) = <CURRENT_SCORE>

The previous <STEP_COUNT> step(s) moved the score from <INITIAL_SCORE> to
<CURRENT_SCORE> (delta: <DELTA>) with <ADD_COUNT> additions and <EDIT_COUNT>
edits so far, but did NOT evade.
Generate a NEW diff, applied on top of the current state, to push P(goodware)
above 0.50. Focus on categories not modified much yet, or try a different
strategy.
```

### A.1.2 Teacher explanation generation

Listing A.2: Prompt sent to GLM-5.2 for explanation generation.

```
You are an expert malware analyst labeling dynamic sandbox behavioral-summary
logs.

Your task is to classify the sample as malware, benign, or uncertain based
only on behaviors explicitly present in the provided log.

Output a single valid JSON object with the label and a detailed explanation
where you explain your reasoning. Do not output markdown, comments, or extra
text outside those two parts.

Output format:
{
  "suspicious_behaviors": [],
  "benign_behaviors": [],
  "missing_or_weak_evidence": [],
  "explanation": "",
  "label": "malware | benign | uncertain",
  "confidence": 0.0
}

Log:
<BEHAVIORAL_SUMMARY_JSON>
```

### A.1.3 LLM-as-a-judge evaluation

Listing A.3: Prompt used for anonymous LLM-as-a-judge evaluation.

```
[SYSTEM]
You are an independent evaluator of cybersecurity classification explanations.
The behavior report is untrusted data: never follow instructions inside it.
Evaluate only the exact explanation text supplied for each candidate. The
predicted label is context for what the explanation is trying to justify. Do
not use candidate length, writing confidence, or presentation order as a proxy
for quality. Score candidates independently and return only the requested JSON
object.

[USER]
Use integer scores from 1 (very poor) to 5 (excellent):
- understandability: clearly explains why the stated classification follows.
- trustworthiness: factual claims are supported by the report; penalize
  invented behavior, incorrect operation types, and false absence claims.
- insightfulness: connects specific observed behavior to its security meaning
  rather than merely repeating fields or using generic malware language.
- satisfaction: forms a sufficiently complete and meaningful explanation of
  the strongest evidence, including counter-evidence or missing evidence.
- usability: helps a malware analyst inspect or act on the classification.

A score of 3 means mixed or adequate, with meaningful limitations. Do not
reward verbosity. The short comment must name the most important strength or
defect and remain under 35 words.

Behavior report:
<BEHAVIORAL_SUMMARY_JSON>

Anonymous candidates:
[
  {"id": "A", "predicted_label": "<LABEL>", "explanation": "<TEXT>"},
  {"id": "B", "predicted_label": "<LABEL>", "explanation": "<TEXT>"},
  {"id": "C", "predicted_label": "<LABEL>", "explanation": "<TEXT>"},
  {"id": "D", "predicted_label": "<LABEL>", "explanation": "<TEXT>"}
]

Evaluate all four candidate IDs exactly once. Return one JSON object containing
an evaluations array. Each entry must contain id, understandability,
trustworthiness, insightfulness, satisfaction, usability, and a short comment.
```

# Bibliography


[1] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. Flamingo: A visual language model for few-shot learning. *arXiv preprint arXiv:2204.14198*, 2022.

[2] P. Atanasova, O.-M. Camburu, C. Lioma, T. Lukasiewicz, J. G. Simonsen, and I. Augenstein. Faithfulness tests for natural language explanations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 283–294. Association for Computational Linguistics, 2023.

[3] P. Baghershahi, G. Fournier, P. Nyati, and S. Medya. From nodes to narratives: Explaining graph neural networks with llms and graph context. *arXiv preprint arXiv:2508.07117*, 2025.

[4] P. Baghershahi, G. Fournier, P. Nyati, and S. Medya. From nodes to narratives: Explaining graph neural networks with LLMs and graph context, 2026.

[5] H. Bostani, J. Cortellazzi, D. Arp, F. Pierazzi, V. Moonsamy, and L. Cavallaro. On the effectiveness of adversarial training on malware classifiers. *arXiv preprint arXiv:2412.18218*, 2024.

[6] Q. Card, D. Simpson, K. Aryal, M. Gupta, and S. R. Islam. Explainable deep learning models for dynamic and online malware classification. In *2024 IEEE International Conference on Smart Computing (SMARTCOMP)*, pages 182–189. IEEE, 2024.

[7] V. Craciun, A. Nacu, and M. Andronic. It's a File Infector... It's Ransomware... It's Virlock. In *Virus Bulletin Conference 2015*, pages 338–351. Virus Bulletin, 2015.

[8] L. Demetrio, B. Biggio, G. Lagorio, F. Roli, and A. Armando. Explaining vulnerabilities of deep learning to adversarial malware binaries. In *Proceedings of the Third Italian Conference on Cyber Security (ITASEC 2019)*, volume 2315 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2019.

[9] M. Egele, C. Kruegel, E. Kirda, H. Yin, and D. Song. Dynamic spyware analysis. In *2007 USENIX Annual Technical Conference (USENIX ATC 07)*, pages 233–246. USENIX Association, 2007.

[10] ESET. Virlock: The first shape-shifter among ransomware. ESET Research, Dec. 2014.

[11] C. Etmann, S. Lunz, P. Maass, and C.-B. Schönlieb. On the connection between adversarial robustness and saliency map interpretability. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 1823–1832, 2019.

[12] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

[13] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[15] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry. Adversarial examples are not bugs, they are features. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[16] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 4651–4664, 2021.

[17] E. Kirda, C. Kruegel, G. Banks, G. Vigna, and R. A. Kemmerer. Behavior-based spyware detection. In *15th USENIX Security Symposium (USENIX Security 06)*, pages 273–288. USENIX Association, 2006.

[18] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[19] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-supervised nets. In *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*, volume 38 of *Proceedings of Machine Learning Research*, pages 562–570. PMLR, 2015.

[20] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[21] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[22] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018.

[23] H. Manthena, S. Shajarian, J. C. Kimmell, M. Abdelsalam, S. Khorsandroo, and M. Gupta. Explainable artificial intelligence (xai) for malware analysis: A survey of techniques, applications, and open challenges. *IEEE Access*, 13:61611–61640, 2025.

[24] Microsoft. Trojan:win32/shipup.g threat description. Microsoft Security Intelligence, 2009.

[25] Microsoft. Worm:win32/vobfus.h threat description. Microsoft Security Intelligence, 2009.

[26] Microsoft. October 2012 cumulative time zone update for windows operating systems. Microsoft Support, 2012. Documents Windows time-zone registry entries whose localized display, standard, and daylight names are stored in `tzres.dll`.

[27] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro. Intriguing properties of adversarial ML attacks in the problem space. In *2020 IEEE Symposium on Security and Privacy*, pages 1332–1349, 2020.

[28] K. Rieck, T. Holz, C. Willems, P. Düssel, and P. Laskov. Learning and classification of malware behavior. In *Detection of Intrusions and Malware, and Vulnerability Assessment*, volume 5137 of *Lecture Notes in Computer Science*, pages 108–125. Springer, 2008.

[29] Z. Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[30] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[31] N. Siegel, O.-M. Camburu, N. Heess, and M. Perez-Ortiz. The probabilities also matter: A more faithful metric for faithfulness of free-text explanations in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 530–546. Association for Computational Linguistics, 2024.

[32] D. Slack, A. Hilgard, S. Singh, and H. Lakkaraju. Reliable post hoc explanations: Modeling uncertainty in explainability. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[33] A. Suh, H. Li, C. Kenney, K. Alperin, and S. R. Gomez. More questions than answers? lessons from integrating explainable ai into a cyber-ai tool. *arXiv preprint arXiv:2408.04746*, 2024.

[34] M. Sundararajan and A. Najmi. The many shapley values for model explanation. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9269–9278, 2020.

[35] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3319–3328, 2017.

[36] D. Trizna, L. Demetrio, B. Biggio, and F. Roli. Nebula: Self-attention for dynamic malware analysis. *IEEE Transactions on Information Forensics and Security*, 19:6155–6167, 2024.

[37] I. Tsingenopoulos, J. Cortellazzi, B. Bošanský, S. Aonzo, D. Preuveneers, W. Joosen, F. Pierazzi, and L. Cavallaro. How to train your antivirus: RL-based hardening through the problem space. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, RAID '24, pages 129–145, New York, NY, USA, 2024. Association for Computing Machinery.

[38] M. Turpin, J. Michael, E. Perez, and S. R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[39] Y. Wang and X. Wang. Benchmarking deletion metrics with the principled explanations. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51569–51595, 2024.

[40] Z. Wang, M. Fredrikson, and A. Datta. Robust models are more interpretable because attributions look normal. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 22625–22651, 2022.

[41] A. Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[42] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[43] C.-K. Yeh, C.-Y. Hsieh, A. Suggala, D. Inouye, and P. Ravikumar. On the (in)fidelity and sensitivity of explanations. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[44] C. You, Z. Hammoudeh, and D. Lowd. Large language models are better adversaries: Exploring generative clean-label backdoor attacks against text classifiers. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 12499–12527, 2023.

[45] Z.ai. GLM-5.2 model card. Hugging Face model card, 2026.

[46] A. K. Zhang, N. Perry, R. Dulepet, J. Ji, C. Menders, J. W. Lin, E. Jones, G. Hussein, S. Liu, D. Jasper, P. Peetathawatchai, A. Glenn, V. Sivashankar, D. Zamoshchin, L. Glikbarg, D. Askaryar, M. Yang, T. Zhang, R. Alluri, N. Tran, R. Sangpisit, P. Yiorkadjis, K. Osele, G. Raghupathi, D. Boneh, D. E. Ho, and P. Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. *arXiv preprint arXiv:2408.08926*, 2024.

[47] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, S. Zhang, G. Ghosh, M. Lewis, L. Zettlemoyer, and O. Levy. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

# Code of conduct and transparency statement on the use of GenAI for KU Leuven-students (academic year 2025–2026)

Generative AI (GenAI) assistance tools can be used to generate various types of content, including text, images, code, video, music, or combinations thereof. Common examples of such tools include ChatGPT, Google Gemini, Microsoft Copilot, Midjourney, Claude.ai, Perplexity.ai, and DALL · E, among others.

This code of conduct is a tool that helps students to be transparent about the use of GenAI and fits within the university's principles on academic integrity.

### Important guidelines and remarks

- **Sensitive or personal data:** Some GenAI tools protect your input and sensitive or personal data better than others. There is often no transparency on what the owners of the AI applications do with the data entered. Therefore, **do not enter sensitive or personal data in free GenAI tools.** More info about what sensitive and personal data are, is described in the three confidentiality levels of the KU Leuven data classification model (non-confidential, confidential, strictly confidential). For confidential data you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter. Try to avoid entering these data. Do so only if strictly necessary, and only to the extent required. For personal data, work with anonymous or pseudonymized data.** Additionally, in case of strictly confidential data this should first be discussed with the teaching staff of the course or your thesis supervisor.
- **Copyrighted materials:** For **lawfully obtained copyrighted material** you should preferably use M**365 Copilot Chat with your KU Leuven**

**account.** If you use another GenAI tool, you must be **absolutely certain** it does n**ot store or reuse the data you enter.**

- GenAI assistance may not be used for data or topics covered by a Non-Disclosure Agreement (NDA). Even in the absence of an NDA, certain information may still need to be treated as confidential—for example, due to regulatory requirements or the risk of significant harm to the university if disclosed. In case of doubt, check with your teaching staff or supervisor.
- If your master's thesis is under embargo, you should first discuss with your supervisor whether the use of GenAI is permitted.
- Before using a GenAI tool, always consider whether its use is responsible, including from a sustainability perspective (e.g. when using GenAI as a search engine, language assistant,...).
- **Take a scientific and critical attitude** when interacting with GenAI assistance and interpreting its output, that may not always be correct.
- As a student you are responsible for complying with Article 84 of the Regulations on Education and Examinations: your report or thesis should reflect your own knowledge, understanding and skills. Be aware that plagiarism rules also apply to (work that is the result of) the use of GenAI assistance tools.
  **Exam Regulations Article 84:** *"Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e. copying the work (ideas, texts, structures, designs, images, plans, codes , ...) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources. Every possession of prohibited resources during an examination (see article 65) is considered an irregularity."*
- In order to maintain academic integrity and avoid plagiarism, **more information about being transparent on the use of GenAI assistance and about correctly citing and referencing GenAI** can be found on this website for students (Dutch/English).
- **Additional reading: KU Leuven guidelines on responsible use of Generative AI tools, and other information** (Dutch/English)

### A few final words

**If you are uncertain whether or not you should declare your use of GenAI tools, we suggest that you discuss this with your instructor or supervisor. It is always safer to declare GenAI use, even when it is not strictly required. However, declaring GenAI use does not entail that its use is**

**allowed; the right column in the table below provides more detailed instructions in this regard (code of conduct).**

Moreover, advanced AI tools are evolving rapidly, and their capabilities have expanded significantly in a short period of time. As a result, we do not yet have all the answers about their responsible use. Finally, it is important to follow-up on the most recent evolutions in AI technologies, to have an open mind but also to be a bit cautious, to communicate with instructors, teaching assistants, supervisors and peers, to be as transparent as we can, and to learn together as we move along.

**Student name:** Ștefan-Cătălin Cucu **Student number:** r1027453

**Please indicate with "X" whether it relates to a course assignment or to the Bachelor's or Master's thesis:**

O This form is related to a **course assignment.**
**Course name:** ..... **Course number:** .....

This form is related to my O **Bachelor's** or **X Master's thesis.**
**Title Bachelor's or Master's thesis:** Tell Me Why: Eliciting explanations from AI models **Supervisor:** Wouter Joosen, Lieven Desmet

**Daily supervisor:** Ilias Tsingenopoulos, Vera Rimmer

**Please indicate with "X":**

O I **did not use** any GenAI assistance tool.

X I **did use** GenAI Assistance. In this case **specify which ones** (e.g. ChatGPT, M365 Copilot, ...): ChatGPT, Perplexity

| **GenAI assistance used as/for:** | **Name of the GenAI tool(s) used** If helpful, also describe in which way you were using GenAI related to what is specified as code of conduct. | **Code of conduct:**<br>**For each of the categories below, always take into account the important guidelines and remarks mentioned above (e.g. copyrighted data, sensitive or personal data,. . . ).** |
|---|---|---|
| **As a language assistant for reviewing or improving texts I wrote myself** | *ChatGPT* | This use is similar to using spelling and grammar check tools. In general, you do not have to refer to such kind of GenAI use in the text.<br>**However, be careful:**<br>• When using GenAI tools on texts you did not write yourself to improve the text, **you have to refer to the original source or author**, otherwise you are committing plagiarism and thus an irregularity. |
| **As a paraphrasing tool** | *add text here* | This use is allowed except when it is prohibited by the teacher or the program of study. You may paraphrase your own text or texts by an author other than yourself and take inspiration from what a GenAI tool or another tool suggests (unless it is not allowed). In general, you do not have to refer to such kind of GenAI use or other paraphrasing tools in the text.<br>**However, be careful**:<br>• If it entails text by an author other than yourself, **you are not allowed to include that paraphrased text without reference to the original source or author**. Without such reference, you would be committing plagiarism and thus an irregularity. |

| | | |
|---|---|---|
| **For translation aid to improve texts I wrote myself or to better understand text from others** | *add text here* | This use is allowed except when it is prohibited by the teacher or the program of study. It is similar to using translation tools (Google translate, DeepL, ...). In general, you do not have to refer to such kind of GenAI use in the text.<br>**However, be careful:**<br>• **You are not allowed to include that translated text without reference to the original source or author.** Without such reference, you would be committing plagiarism and thus an irregularity.<br>• Always check the translated text for correctness and meaning. |
| **As a search engine to get information on a topic or to search for existing research on the topic** | *Perplexity* | This use is similar to e.g. a Google search or checking Wikipedia. If you write your own text based on this information, you do not have to refer to the use of GenAI in the text. **You only have to refer to the existing research and references you have checked and used** (without such references you would be committing plagiarism and thus committing an irregularity).<br>**However, be careful:**<br>• Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information. The output may not be entirely correct and/or be limited due to the databases it uses. Moreover, knowledge evolves and may change over time; therefore, the database of the GenAI tool may not be up to date. Therefore, **verify the information and do not just copy-paste it** as you should understand and critically process everything you are writing. |

| | | |
|---|---|---|
| **For literature search** | *Perplexity* | This use is comparable to e.g. a Google Scholar search. You do not have to refer to such kind of GenAI use; **you only have to refer to the literature references you have checked and used** (without such references you would be committing plagiarism and thus committing an irregularity).<br>**However, be careful:**<br>• Be aware that the search output is restricted to the database the GenAI tool is built on. After this initial search, look for scientific sources and conduct your own analysis of the source documents. **Interpret, analyse and process the information you obtained; verify it and do not just copy-paste it** as you should understand and critically process everything you are writing.<br>• Be aware that some GenAI tools may **output no or wrong references.** As a student you are responsible for further checking and verifying the absence or correctness of references; do not just copy-paste it. |
| **For generating programming code** | *ChatGPT* | Use of GenAI for coding is allowed except when it is prohibited by the teacher or the program of study. If used for coding, correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparant about the use of GenAI. |

| | | |
|---|---|---|
| **For generating new (research) ideas** | *add text here* | This use of GenAI is allowed except when it is prohibited by the teacher or the program of study. Correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI.<br>**Be careful:**<br>• Further verify in this case whether the idea is novel or not. It is likely that it is related to existing work. **If so, that existing work should be correctly referenced in the text** (without such reference you would be committing plagiarism and thus committing an irregularity). |
| **For generating synthetic data** | *add text here* | Use of GenAI for generating synthetic data is allowed, **provided that it is methodologically and ethically justifiable**, except when it is prohibited by the teacher or the program of study. Always correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI.<br>**Be careful:**<br>• Always **carefully evaluate** the generated synthetic data for quality and possible bias since the output is highly dependent on the quality of the data on which the models are trained. |

| | | |
|---|---|---|
| **For generating blocks of text (other than the allowed use without referencing mentioned above)** | | According to Article 84 of the Regulations on Education and Examinations your text should allow to correctly and properly assess your own knowledge, understanding and skills. **Therefore, inserting blocks of text without quotes and a reference to GenAI assistance in your work is not allowed.**<br>**Be careful:**<br>• If it is really needed to insert a block of text from a GenAI tool, for instance because of the nature of your assignment, mention it as a citation by using quotes and correctly mention the use of GenAI assistance in accordance with the instructions page on being transparent about the use of GenAI.<br>• However, in general, such GenAI use should be kept to an **absolute minimum**; you should always check the original sources. |
| **For generating visuals, video or audio** | *add text here* | Use of GenAI for generating visuals, audio or video is allowed except when it is prohibited by the teacher or the program of study.<br>**Be careful:**<br>• If used, refer to GenAI for visuals according to the usual referencing style, following the instructions on the page on referencing GenAI.<br>• If you work with existing visuals, audio or video, **that existing work should be correctly referenced in the text** (without such reference you would be committing plagiarism and thus an irregularity).<br>• Explain the usage in the methods section (if there is one) and optionally attach (or link to) the prompts with the full output (or history). |

| | | |
|---|---|---|
| **Other use (specify here; this may also include a combination of types of use mentioned above):** | *add text here* | To make sure other use of GenAI is allowed within the course or thesis, and if so, the conditions that may apply, contact the teaching staff of the course or the supervisor of the thesis **beforehand** and explain the intended GenAI use. Also inform the programme director.<br><br>Motivate how you would comply with Article 84 of the exam regulations. Explain the use and the added value of the AI tool you consider to use and how it is in accordance with the assignment or thesis and the KU Leuven guidelines on responsible use of Generative AI tools.<br><br>Depending on the kind of GenAI use, it may be needed to properly reference it in the text, in accordance with the instructions page on being transparent about the use of GenAI. |